COLLEGE OF COMPUTER STUDIES

Information Management 2 (IM2) — Advance Database System

FINAL PROJECT

SUBMISSION PAPER TEMPLATE

(includes Data Flow Diagram, System Flowchart, and Database Schema)

Project Title	*Micro Retail Business Point of Sales (Web-Based - Mobile/Desktop)*
Student Name	Borlaza, Lorenz James S.
Course / Year & Section	BSIT-B - LFCA311N115
Programming Language Used	(HTML, CSS, JAVASCRIPT, PYTHON,
Database Platform	XAMPP (MySQL/MariaDB)
Instructor	FRANCISCO OCSAN
Date Submitted	November 6, 2026

Table of Contents

I. Introduction

II. Data Flow Diagram (DFD)

III. System Flowchart

IV. Database Design and Schema

V. User Interface (UI) and User Experience (UX) Design

VI. Security and Backup Documentation

VII. Individual Reflection (Data Privacy and Ethics)

VIII. References

Appendix A — Source Code Repository / Submission Link

Appendix B — Approval Sheet

I. Introduction

A. Background of the Study

The Micro-Retail Point of Sales (POS) system is designed specifically for small-scale businesses and neighborhood sari-sari stores transitioning from manual logging to a secure, digital platform. In many local communities, these micro-retailers act as essential economic lifelines. However, most still depend on paper notebooks to record sales, stock levels, and customer credit—a process prone to human error, lost records, and untracked debts that directly undermine profitability. Furthermore, compiling fragmented paper records for annual financial reviews creates significant operational stress and hides true business performance.

To overcome these challenges which are faced by micro-retailers, I developed this system to make an accessible, centralized management tool tailored to their daily operations. Manual credit tracking frequently leads to financial loss and customer friction due to the lack of real-time balance updates during checkout or repayment. By adopting this web-based, mobile-responsive POS system, owners can replace vulnerable paper logs with a reliable digital infrastructure. This system automates inventory tracking, streamlines debt and credit management, and delivers real-time dashboard summaries and exportable reports that ultimately protect business data and maximize daily profitability.

B. Objectives

- **General Objective:**

  To develop and implement an micro-retail business Point-of-Sale (POS) System for small sari-sari store owners that improves the efficiency, accuracy, and management of sales transactions and inventory operations.
- **Specific Objective 1:** To develop a micro-retail business Point-of-Sale (POS) system for fast and accurate sales transactions and debts recording.
- Specific Objective 2: To create a system that monitors product stock, debts, and sales.
- Specific Objective 3:3. To automatically update inventory whenever a sale and transactions is completed

C. Scope and Limitations

Scope:

- The project only covers a web-based, responsive Point of Sales (POS) system optimized for mobile and desktop viewports, specifically tailored for small-scale sari-sari stores and micro-retailers. The system includes secure user authentication, a dashboard summarizing daily sales, debts, and inventory levels, a product management module supporting custom product codes, and a transactional checkout interface that processes cash, full debt, and partial payments. Additionally, it features a dedicated customer debt tracker, a sales log for completed cash transactions, a comprehensive system-wide audit history log, and a settings panel capable of CSV/Excel data export and annual system resets.

Limitations:

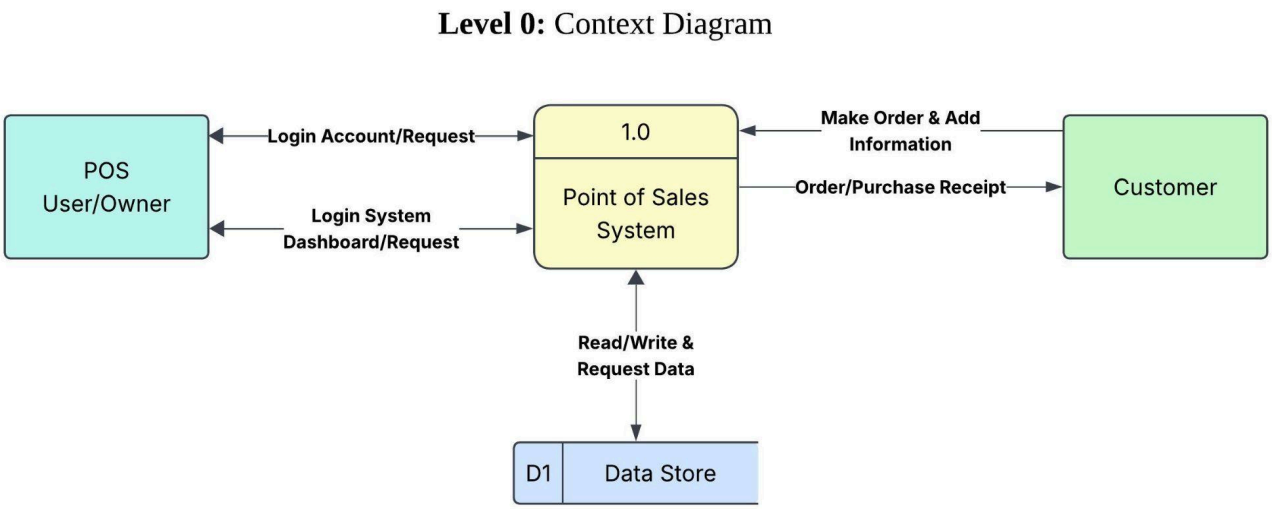
- While the limitation is that the system operates strictly as a single-user application without multi-branch synchronization, barcode scanner integration, or automatic tax/receipt printing capabilities. Furthermore, it does not support real-time digital wallet or credit card integrations (e.g., GCash, Maya), offline data caching (requiring an active internet connection to process sales), or automatic supplier reordering alerts when stocks run low.

II. Data Flow Diagram (DFD)

A Data Flow Diagram shows how data moves through your system — the external entities that send or receive data, the processes that transform it, and the data stores (your database tables) where it is kept.

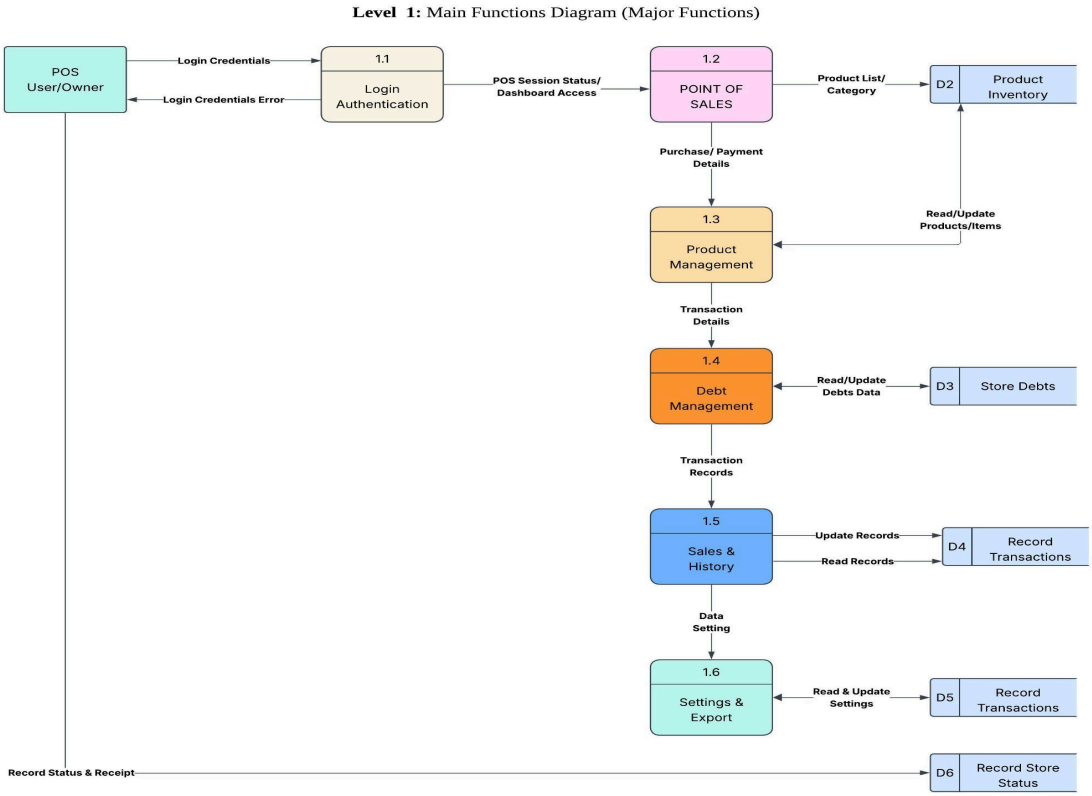
A: Context Diagram (Level 0)

Figure 1: Context Diagram (Level 0)



B: Level 1 DFD (Process Breakdown)

Figure 2: Level 1 (DFD (Process Breakdown)



C. DFD Process Description

Table 1: DFD Precess Description

No.	Process Name	Input	Output	Data Store(s) Used
1.0	Point of Sales system	Make order and add information, Login Account/ Request	Order/purchase receipt, Login System Dashboard board	Customer and Owner information
1.1	Login Authentication	Login Credentials from (owner/ customer)	Login Credentials error from (owner/ customer)	Customer and Owner information
1.2	POINT OF SALES	POS Session Status / Dashboard Access	Purchase/ Payment Details	Product Inventory
1.3	Product Management	Purchase/ Payment Details	Transaction Details, and Read/Updates Products/Items	Product Inventory
1.4	Debt Management	Transaction Details	Transaction Records, and Read Update Debts Data	Store Debts
1.5	Sales & History	Transaction Records & Read Record	Update Records & Data setting	Record transactions
1.6	Settings & Export	Data Setting	Read & Update settings, and Record Status & Receipt.	Record transactions, & Record Store Status.

III. System Flowchart

The flowchart shows the step-by-step logic of your system's main operation (for example, the login-to-transaction flow, or the record-processing flow) using standard flowchart symbols: Start/End (oval), Process (rectangle), Decision (diamond), and Input/Output (parallelogram).

A. Main System Flowchart

Figure 2: Main System Flowchart

B. Flowchart Narrative

Briefly narrate the flow in your own words, step by step, matching the diagram above.

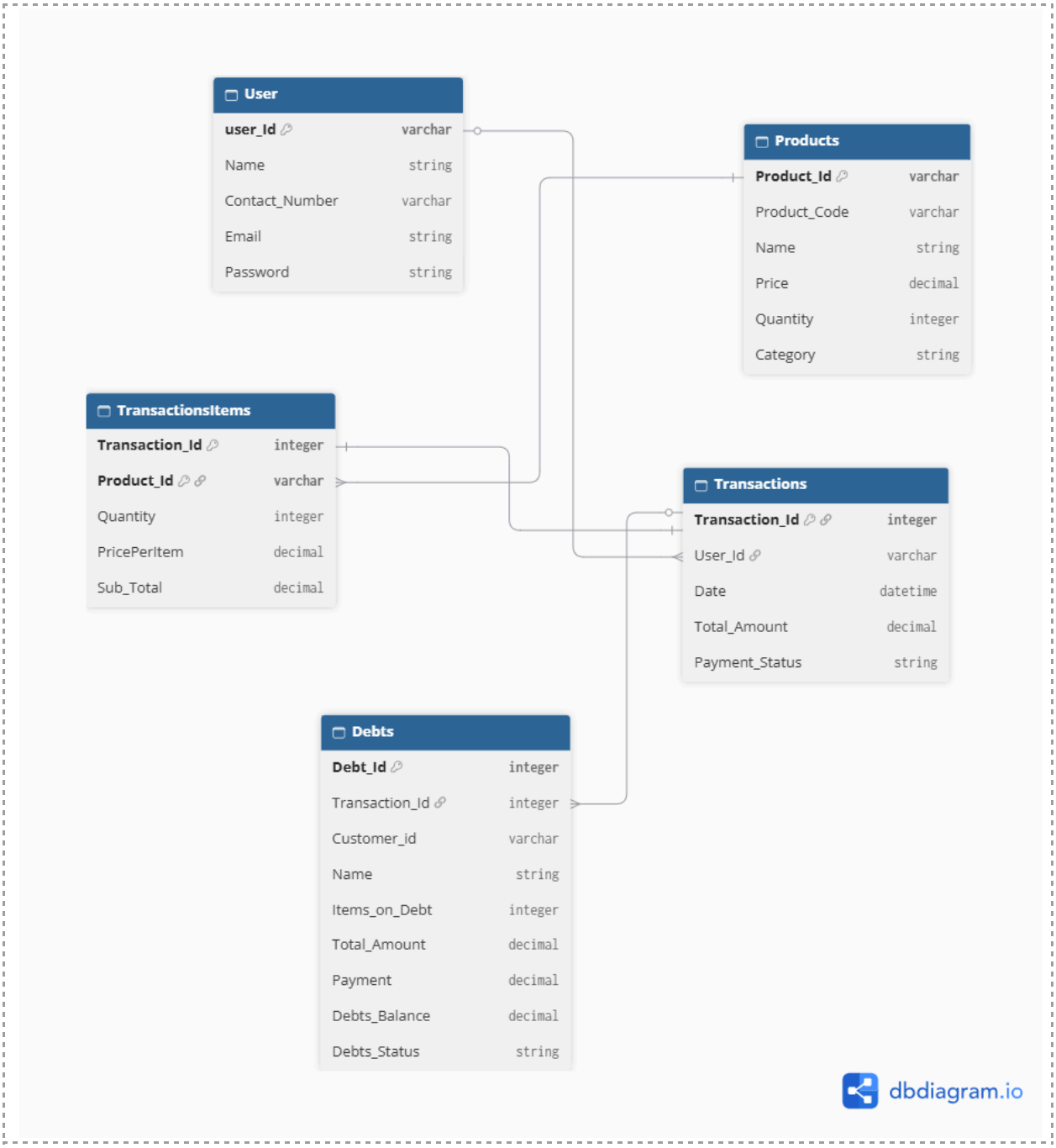
The flowchart above clearly shows the process of creating an account, saving account created to the database, changing passwords, it also shows the step by step process on the main system, that how each pages and menus connecting to each other and exchanging information and recording on their specific pages and functions, also the flowchart shows how information gathered through importing, exporting, and backup if needed, and it also shows the step by step process of logging out from the system, and additionally, it shows how the login and signup system processes, like when a user or admin opens the system and click ok, a modal state show, the validation before proceeding, so if the user or admin don't have an account a validation state appears you dont have an account yet, which has a decision, second if the user already have an account registered, and login validation appears before proceeding to login dashboard, and before the main-dashboard.

IV. Database Design and Schema

A. Entity-Relationship Diagram (ERD)

Paste your ERD below — either exported from phpMyAdmin's Designer tab, or drawn in draw.io/Lucidchart. It must show all tables, primary/foreign keys, and relationship lines (1:1, 1:M, M:M).

Figure 3: Entity-Relationship Diagram (ERD)



B. List of Entities and Relationships

Table 2: List of Entities and relationships

Entity (Table)	Description	Related To	Relationship Type
User	The user entity(table) will collect all the user's information added during the sign up and login process.	Transactions	One-to-Many (1:N)

Products	Products Will store all the products information that is added on the system.	TransactionItmes	One-to-Many (1:N)
Transactions	Will Collect all the transaction information of the user or customer such as name, contact number, items, etc.	TransactionItmes, Debts	One-to-Many (1:N)
TransactionItmes	Acts as an associative (junction) entity that bridges transactions and products. It records the individual line items included in a specific checkout session, storing crucial details such as the quantity purchased, unit price at the time of sale, and subtotal for each unique product linked to a transaction.	Transaction	One-to-One (1:1)
Debts	Manages customer credit and accounts receivable for the micro-retail store. It tracks outstanding balances, credit limits, repayment histories, and due dates for customers who purchase items on credit rather than paying in full upfront.	Transactions, user	One-to-zero-one (1:0..1)

Add more rows as needed — the project requires at least 7 tables.

C. Normalization Summary

Briefly show how one representative table evolved from Unnormalized Form (UNF) to at least 3NF/BCNF. You may attach the full walk-through as an appendix if it is long.

Table 3: The Normalization Summary

UNF	**TransactionRecord** (transaction_id, date, user_id, user_name, customer_name, customer_contact, { product_id, product_name, unit_price, quantity_purchased, line_subtotal }, total_amount, payment_type, items_on_debt) *Explanation:* It Contains repeating groups (multiple items inside curly braces {}) and un-atomic attributes combined into a single flat paper receipt log.
1NF	**TransactionRecord_1NF** (*transaction_id*, *product_id*, date, user_id, user_name, customer_name, customer_contact, product_name, unit_price, quantity_purchased, line_subtotal, total_amount, payment_type, items_on_debt)

	Explanation: The repeating groups is remove. Which created a unique composite primary key ($\underline{transaction_id}, \underline{product_id}$). All attribute cells now hold atomic (single) values.
2NF	**1. Transactions** ($\underline{transaction_id}$, date, user_id, user_name, customer_name, customer_contact, total_amount, payment_type, `items_on_debt`) **2. TransactionItems** ($\underline{transaction_id}, \underline{product_id}$, product_name, unit_price, quantity_purchased, line_subtotal) *Explanation:* It removed the **partial dependencies**. Attributes depending only on transaction_id (e.g., date, customer_name) it separated from attributes depending on both transaction_id and product_id (e.g., quantity_purchased).
3NF / BCNF	**1. User** ($\underline{user_id}$, user_name, role, password_hash) **2. Products** ($\underline{product_id}$, product_name, unit_price, stock_qty) **3. Transactions** ($\underline{transaction_id}$, date, $user_id^*$, customer_name, customer_contact, total_amount, payment_type) **4. TransactionItems** ($\underline{item_id}$, $transaction_id^*$, $product_id^*$, quantity_purchased, unit_price_at_sale, line_subtotal) **5. Debts** ($\underline{debt_id}$, $transaction_id^*$, outstanding_balance, due_date, status) *Explanation:* The **transitive dependencies** are removed. Attributes non-transitively dependent on primary keys (like user_name depending on user_id, product_name depending on product_id, and debt specifics depending on payment outcome) were extracted into their respective independent tables linked via Foreign Keys (FK^*).

D. Data Dictionary

Copy the table below once for every table in your database (at least 7 copies). Rename the heading to match each table name.

Table Name: user

Table 4.0: The User Table

Field Name	Data Type	Size	Key / Constraint	Description
user_id	varchar(100)	100	Primary Key	Uniquely identifies each user row.
Name	text	Empty	Not Null	Ensures every user profile has a registered name.
Contact Number	varchar(100)	100	Null or Unque	Allows optional contact numbers or enforces uniqueness to prevent duplicate entries.
Email Address	text	Empty	Unique or Not Null	Prevents duplicate user accounts and ensures valid login identifiers.
Password	text	Empty	Not Null	Required for authentication security.

(Duplicate the table and heading above for each remaining table: table 2, table 3, and so on.)

Table Name: transactionsitems

Table 4.1: The Transction Items Table

Field Name	Data Type	Size	Key / Constraint	Description
Transaction_id	int	100	PK, FK	Links line items to a specific transaction in the `Transactions` table. Forms part of the composite primary key.
Product_id	varchar	100	PK, FK	Links the line item to a specific product in the `Products` table. Forms part of the composite primary key.
Quantity	int	100	Not Null	Specifies the number of units purchased for this item. Must not be empty.
PricePerItem	Decimal	62, 2	Not Null	Stores the historical unit price of the product at the exact time of sale.

Sub_Total	Decimal	62, 2	Not Null	Stores the calculated total for this line item ($Quantity \times PricePerItem$).

Table Name: transactions

Table 4.2: The Transction Table

Field Name	Data Type	Size	Key / Constraint	Description
Transaction_id	int	100	PK, FK	Uniquely identifies each transaction record
user_id	varchar	100	PK, FK	`User(user_id)` to identify which system user/cashier processed the checkout
Date	datetime	6	Not Null	Timestamp recording when the transaction was executed
Total_Amount	decimal	62, 2	Not Null	Stores the total overall cost of all items in the transaction
Payment_Status	text	empty	Not Null	Indicates the status of the transaction (e.g., 'Paid', 'Pending', 'Partial').
Paid_Amount	decimal	62, 2	Not Null	Stores the actual cash/amount tendered by the customer during checkout.
Payment_Type	varchar	20	Not Null	Identifies the payment classification (e.g., 'Cash', 'Full Debt', 'Partial Debt').
Payment_Method	varchar	50	Not Null	Stores the payment channel used (e.g., 'Cash', 'GCash', 'Maya')

Customer_Name	varchar	150	NUL	Stores the customer's name (optional for cash sales, required for debt transactions)

Table Name: product

Table 4.3: The product Table

Field Name	Data Type	Size	Key / Constraint	Description
Product_id	int	100	PK, FK	Uniquely identifies each product record in the store inventory
Product_Code	varchar	100	Unique, Not Null	Unique barcode, SKU, or custom code assigned to identify the product
Name	text	150	Not Null	Stores the descriptive title or name of the product item
Price	decimal	62, 2	Not Null	Stores the standard selling unit price of the item
Quantity	int	100	Not Null	Tracks the current available stock quantity in inventory.
Category	text	100	Not Null	Classifies the product into a grouping (e.g., 'Beverages', 'Snacks', 'Canned Goods')
Low_Stock_Threshold	int	20	Not Null	Minimum stock level that triggers a low-inventory warning alert.
Id_Active	tinyint	1	Not Null	Soft-delete flag (e.g., `1` for active, `0` for archived/discontinued products)

Table Name: pos_settings

Table 4.4: The Pos SettingTable

Field Name	Data Type	Size	Key / Constraint	Description
Setting_Key	varchar	100	PK	Uniquely identifies each configuration setting key (e.g., `'store_name'`, `'receipt_header'`, `'currency_symbol'`).
Setting_Value	varchar	255	Not Null	Stores the corresponding configuration value assigned to the setting key

Table Name: pos_id_sequence

Table 4.5: The Pos Id Sequence Table

Field Name	Data Type	Size	Key / Constraint	Description
Sequence_name	varchar	32	PK	Uniquely identifies the sequence counter name (e.g., `'transaction_id'`, `'product_code'`).
latest_id	bigint	20	Not Null	Stores the latest numerical value generated for the sequence to track incremental IDs.

Table Name: debt_transaction

Table 4.6: The Debt_Transaction Table

Field Name	Data Type	Size	Key / Constraint	Description
Debt_id	bigint	20	PK, FK	Links the payment or record to a specific customer debt profile (`debts.debt_id`).
Transaction_id	bigint	20	FK, Not Null	Links the transaction where credit or repayment was recorded (`transactions.transaction_id`).
Linked_At	datetime		Not Null	Timestamp recording the exact date and time when the

				transaction and debt were linked or when a payment was applied.

Table Name: debt_payment

Table 4.7: The Debt_Payment Table

Field Name	Data Type	Size	Key / Constraint	Description
Payment_id	bigint	20	PK	Uniquely identifies each debt repayment transaction record.
Debt_id	bigint	20	FK	References `debts(debt_id)` to specify which customer debt account is being paid.
Transaction_id	bigint	20	Not Null	Links the payment to the corresponding transaction record in the `transactions` table.
Amount	decimal	62, 2	Not Null	Stores the total monetary value of the debt payment made by the customer.
Paid_At	datetime		Null	Timestamp recording the exact date and time when the debt payment was received.

Table Name: debts

Table 4.8: The Debts Table

Field Name	Data Type	Size	Key / Constraint	Description
Debt_id	int	100	FK	Uniquely identifies each customer debt profile record
Transaction_Id	int	65	PK	References `transactions(transaction_id)` to link the debt to its original transaction .
Customer_Id	varchar	100	FK	References the customer profile or account identifier

Name	text		Not Null	Stores the full name of the customer owing the debt.
Items_on_debt	int	100	Not Null	Tracks the total quantity or count of items purchased on credit.
Total_amount	decimal	65, 2	Not Null	Stores the total initial credit value for the transaction
Payment	decimal	65, 2	Not Null	Tracks the cumulative monetary amount paid toward the debt.
Debts_Balance	decimal	65, 2	Not Null	Stores the current remaining unpaid balance ($Total_amount - Payment$).
Debts_Status	text		Not Null	Indicates the debt status (e.g., 'Unpaid', 'Partially Paid', 'Fully Paid').
Is_Archived	tinyint	1	Not Null	Soft-delete flag (1 if archived/settled, 0 if active).

V. User Interface (UI) and User Experience (UX) Design

A. Wireframe / Mockup (created before development)

Figure 4.0: The User Interface(UI) & User Experience (UX)

Figure 4.1 The Welcom Page

Figure 4.1 The Welcom Page

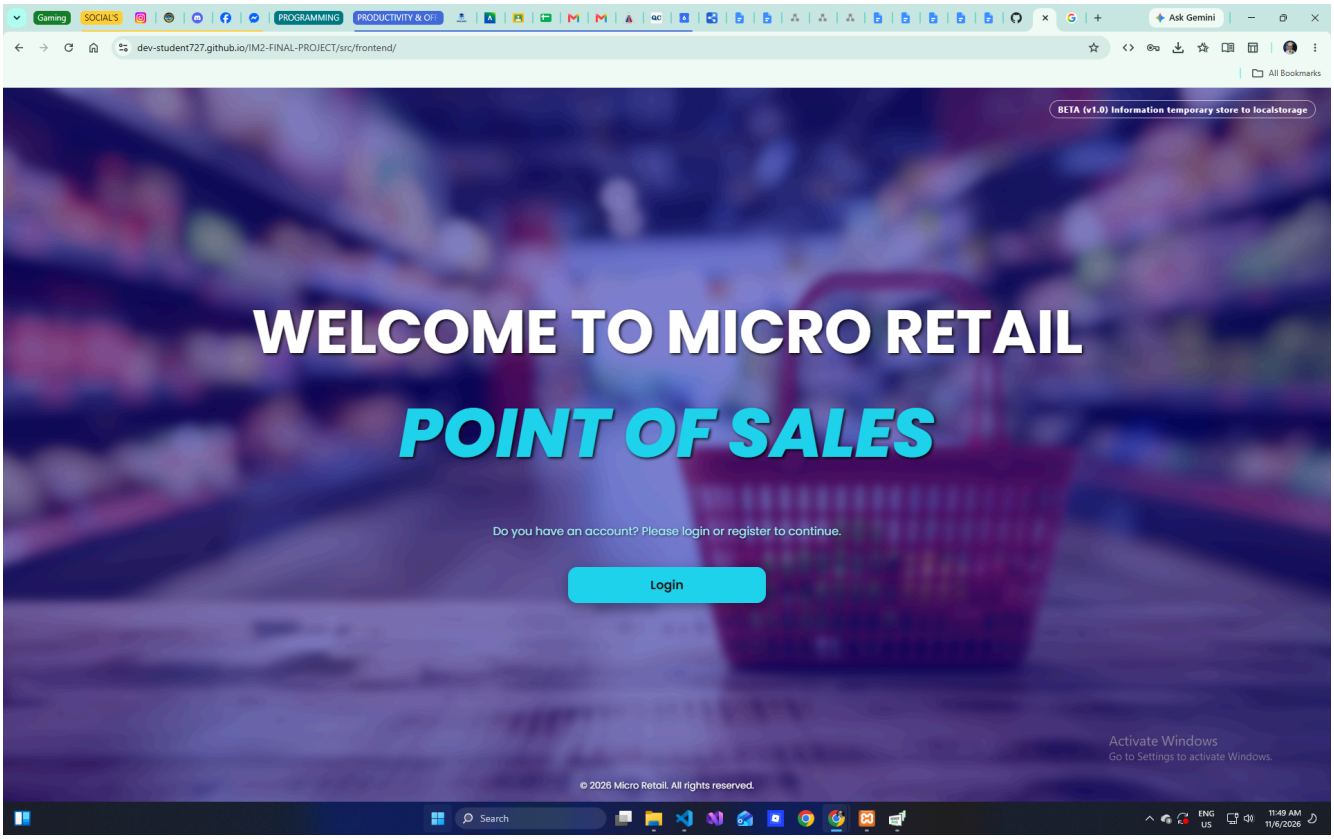


Figure 4.2: The Modal State

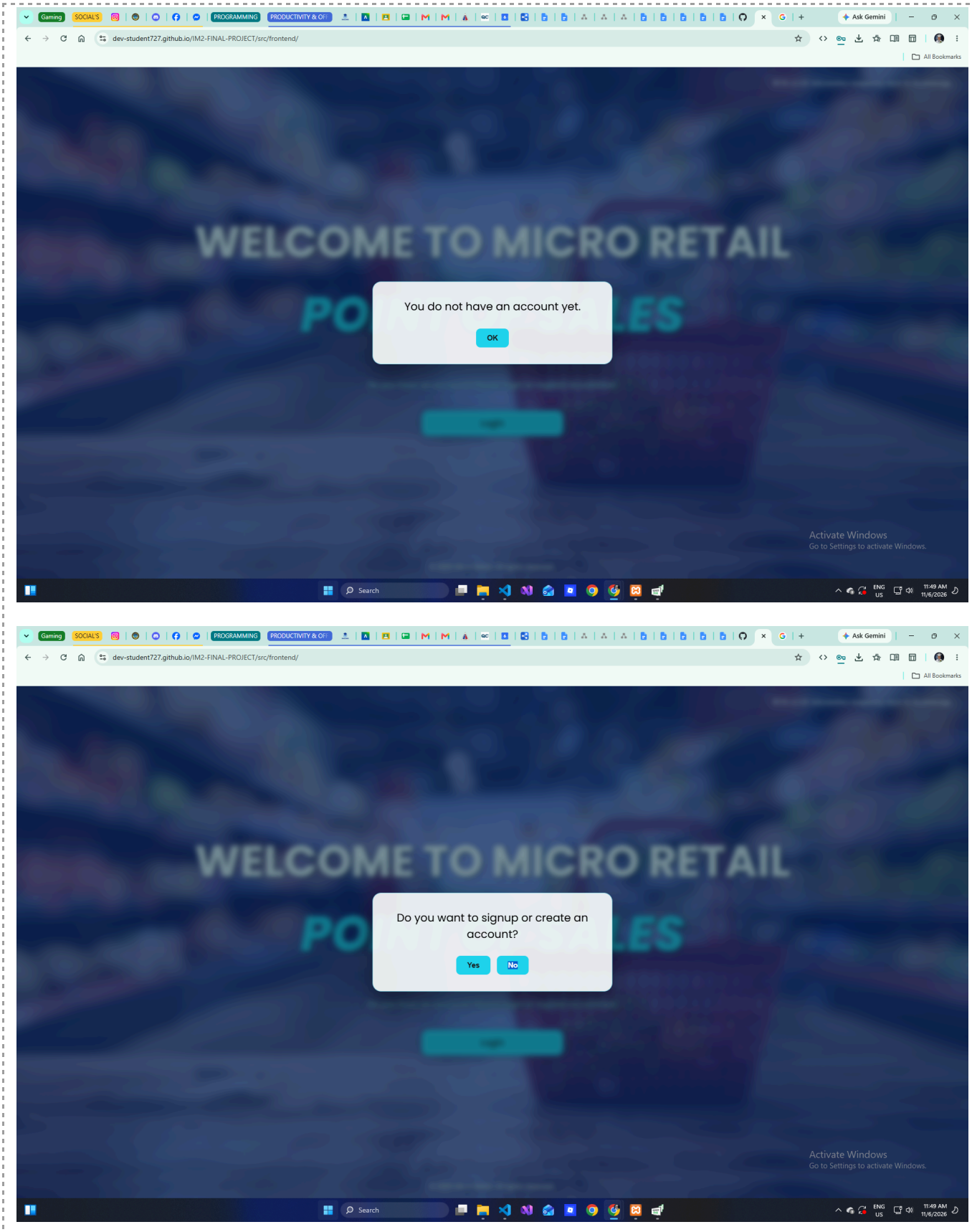
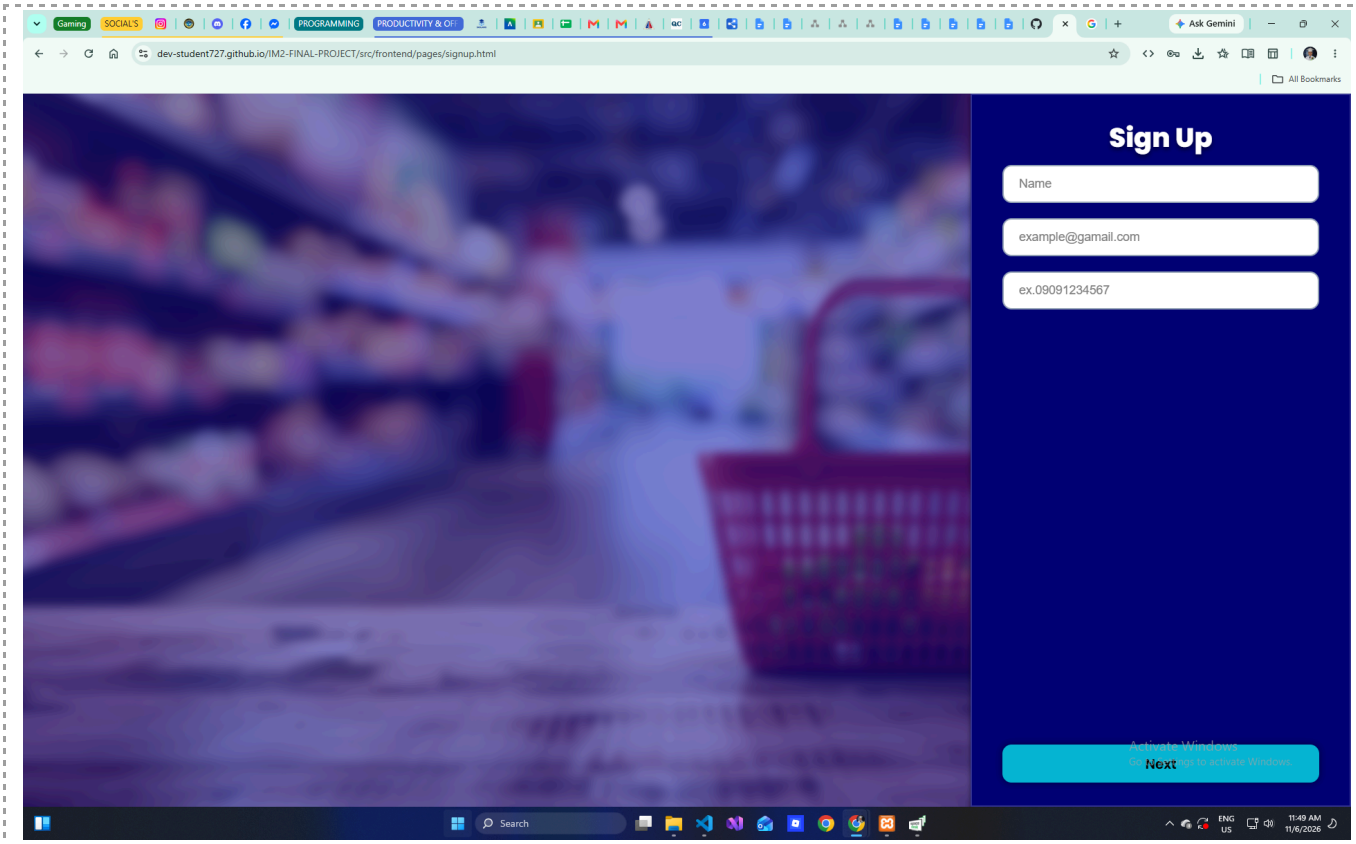
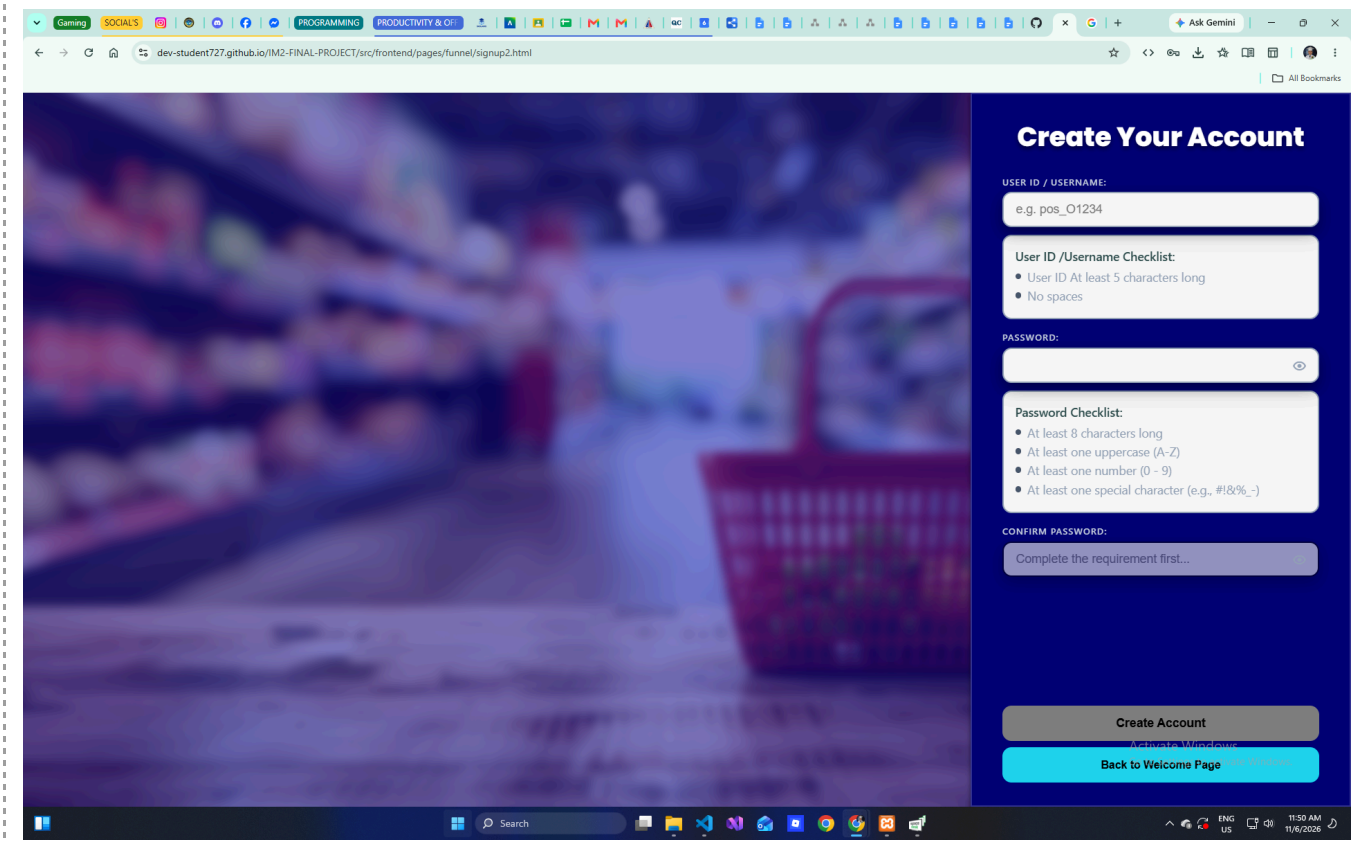
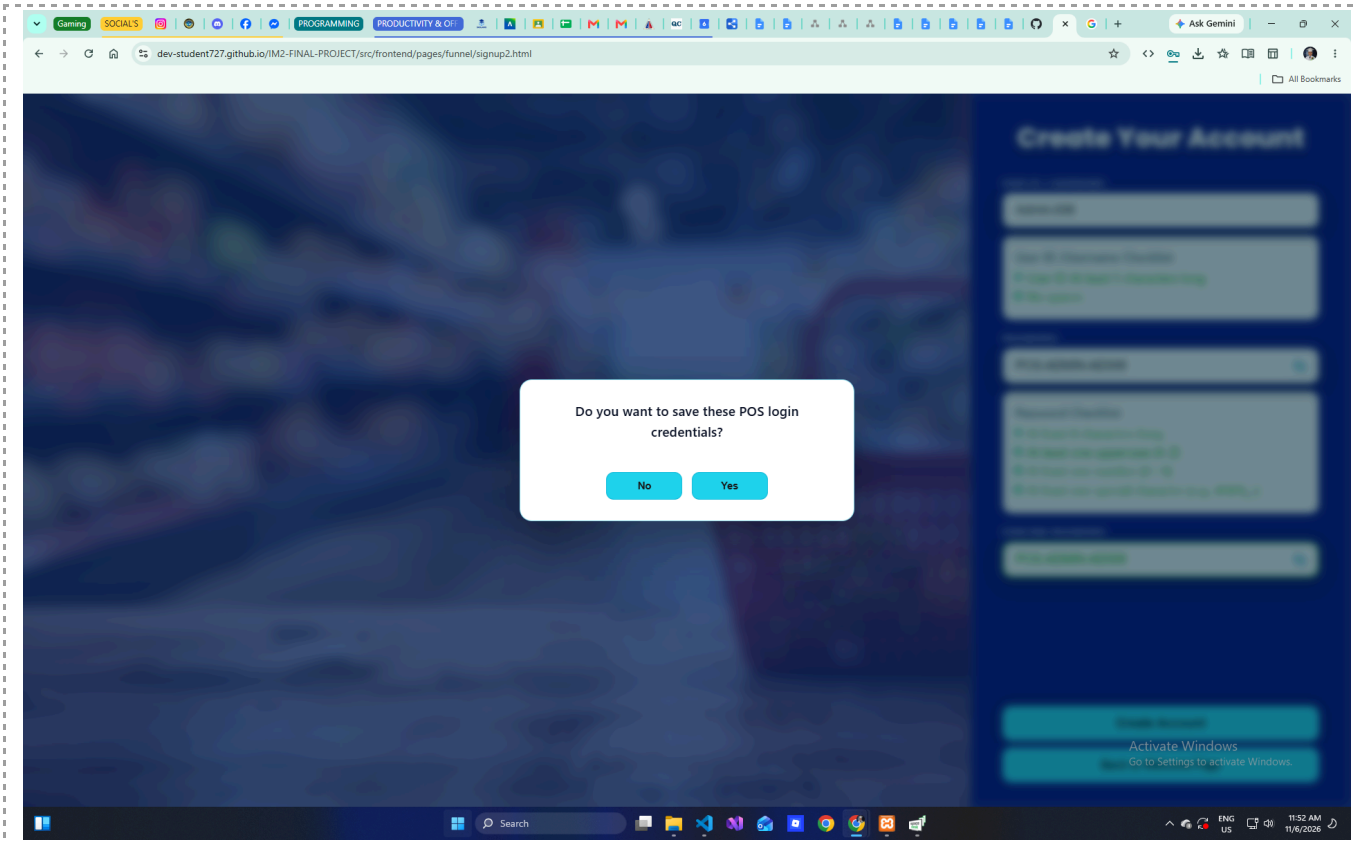
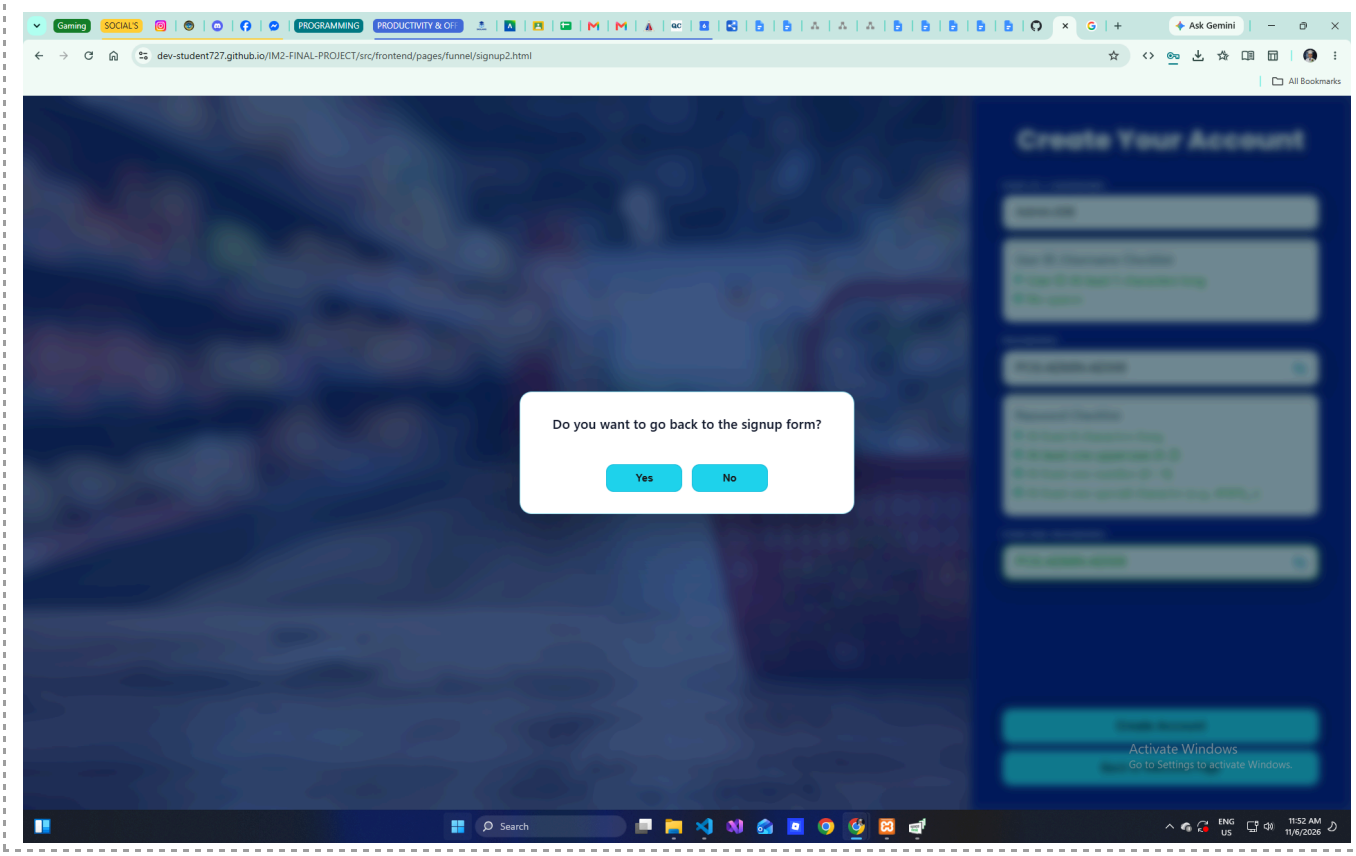


Figure 4.3: Signup page and account creation


dev-student727.github.io/IM2-FINAL-PROJECT/src/frontend/pages/signup.html
Sign Up
Name
example@gamail.com
ex.09091234567
Next


dev-student727.github.io/IM2-FINAL-PROJECT/src/frontend/pages/funnel/signup2.html
Create Your Account
USER ID / USERNAME:
e.g. pos_O1234
User ID /Username Checklist:
User ID At least 5 characters long
No spaces
PASSWORD:
Password Checklist:
At least 8 characters long
At least one uppercase (A-Z)
At least one number (0 - 9)
At least one special character (e.g., #!&%_-)
CONFIRM PASSWORD:
Complete the requirement first...
Create Account
Back to Welcome Page


dev-student727.github.io/IM2-FINAL-PROJECT/src/frontend/pages/funnel/signup2.html
Create Your Account
Do you want to save these POS login credentials?
No
Yes
Activate Windows
Go to Settings to activate Windows.


dev-student727.github.io/IM2-FINAL-PROJECT/src/frontend/pages/funnel/signup2.html
Create Your Account
Do you want to go back to the signup form?
Yes
No
Activate Windows
Go to Settings to activate Windows.

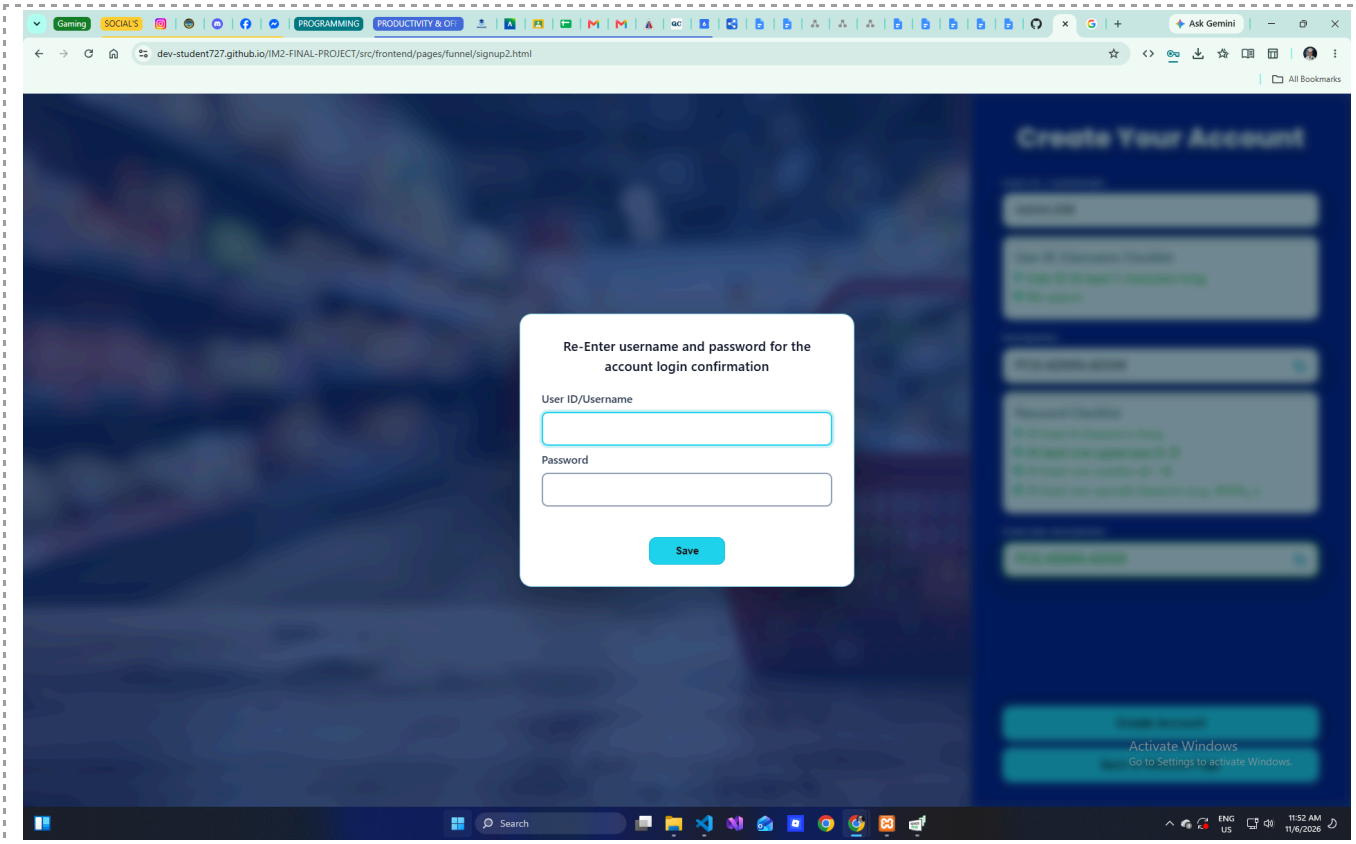


Figure 4.4: Dashboard

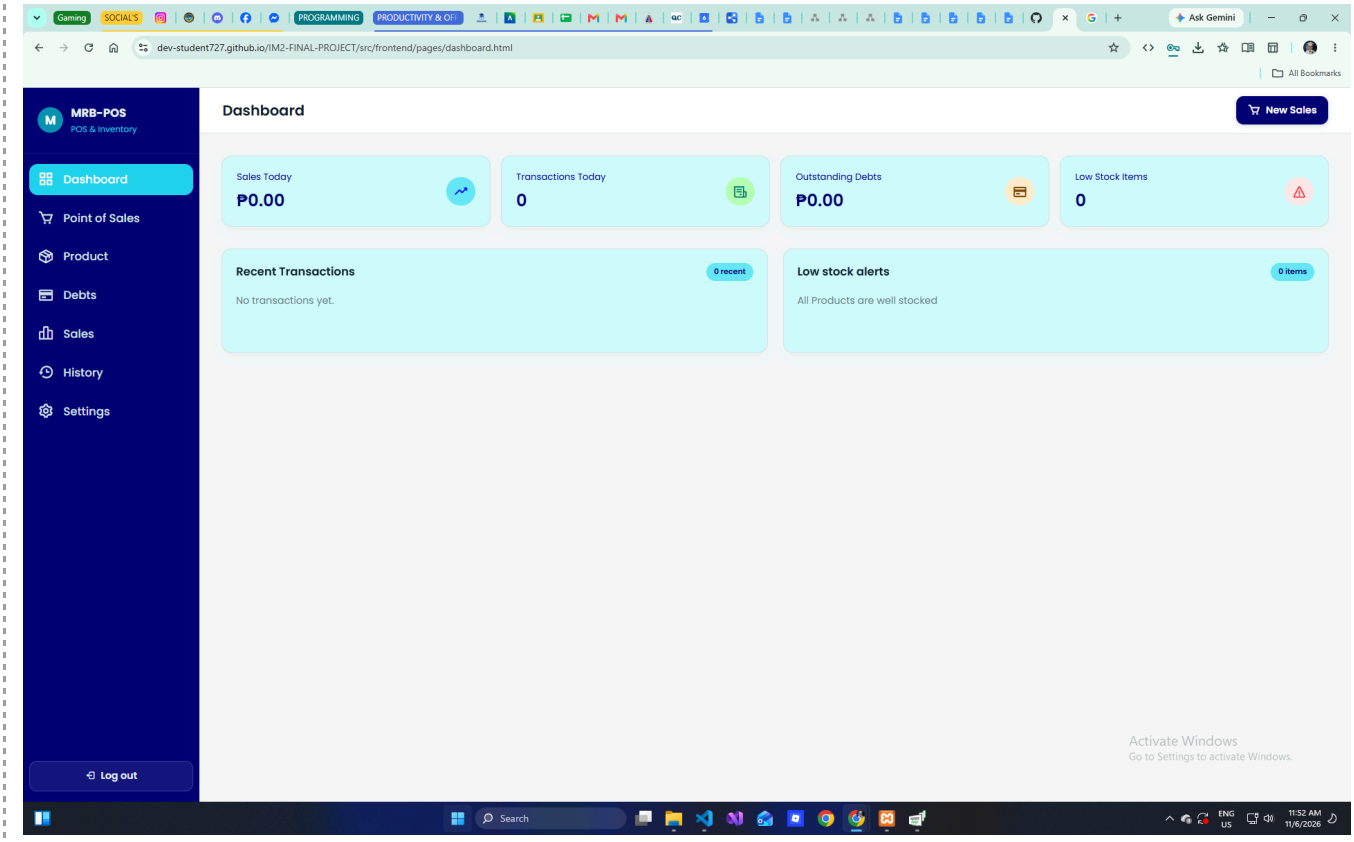


Figure 4.5: Point of Sales

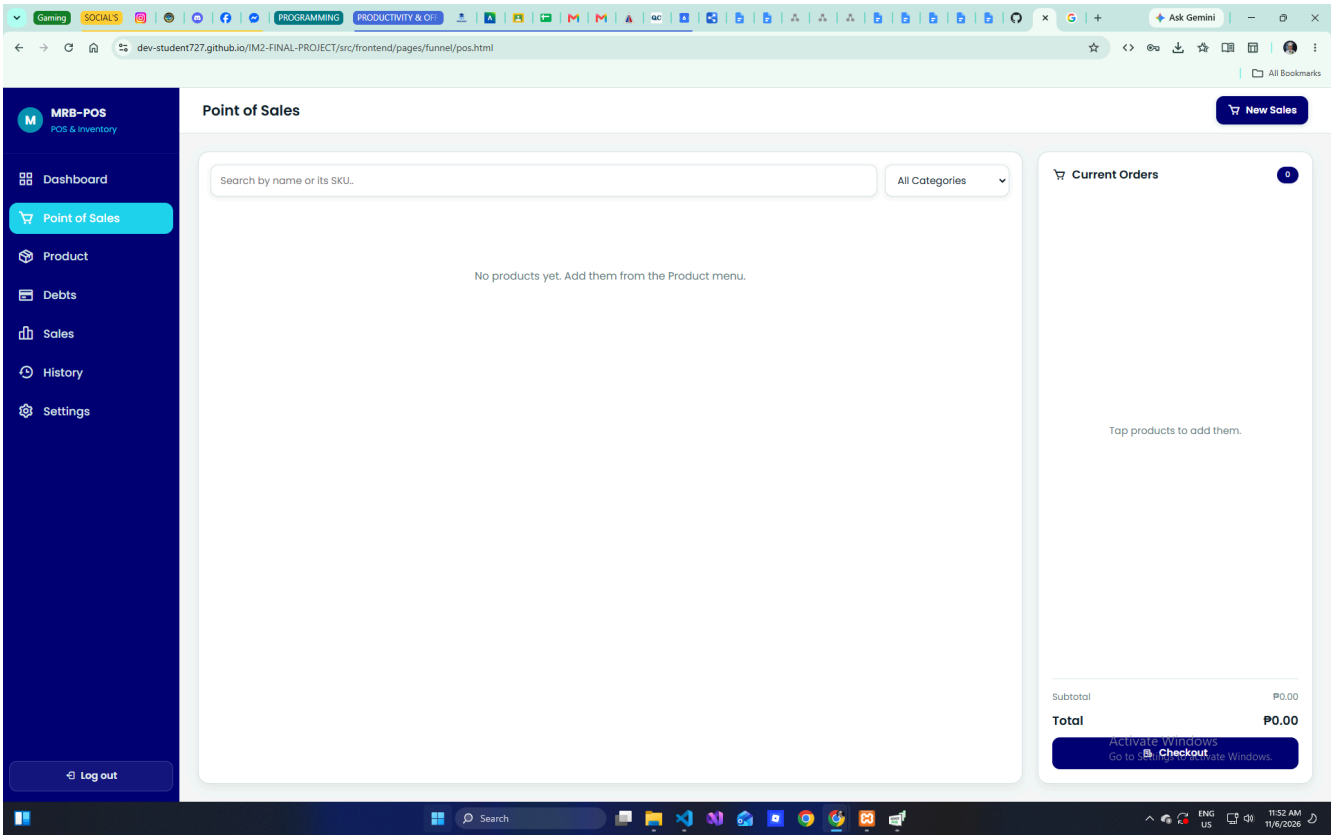


Figure 4.6: Products

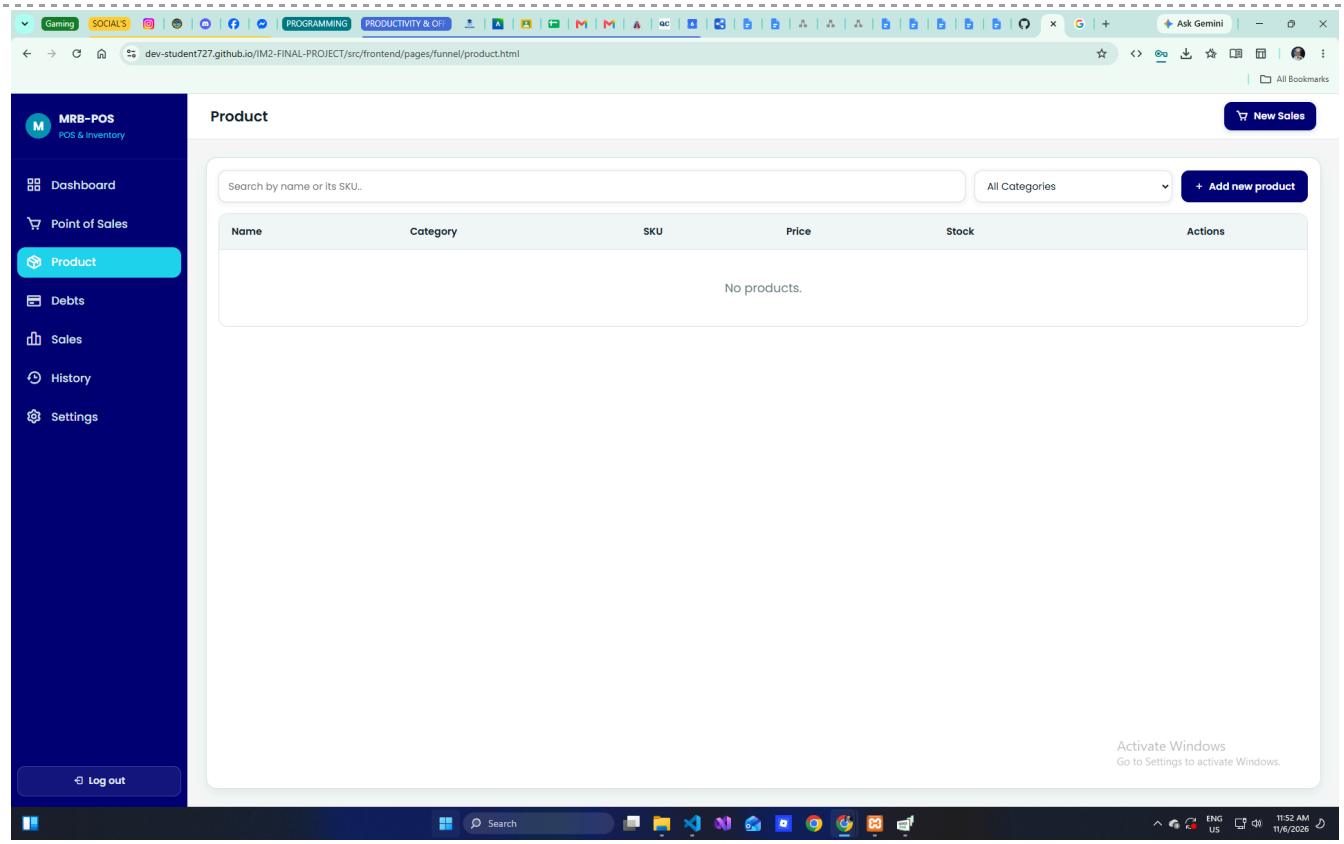


Figure 4.7: Debts

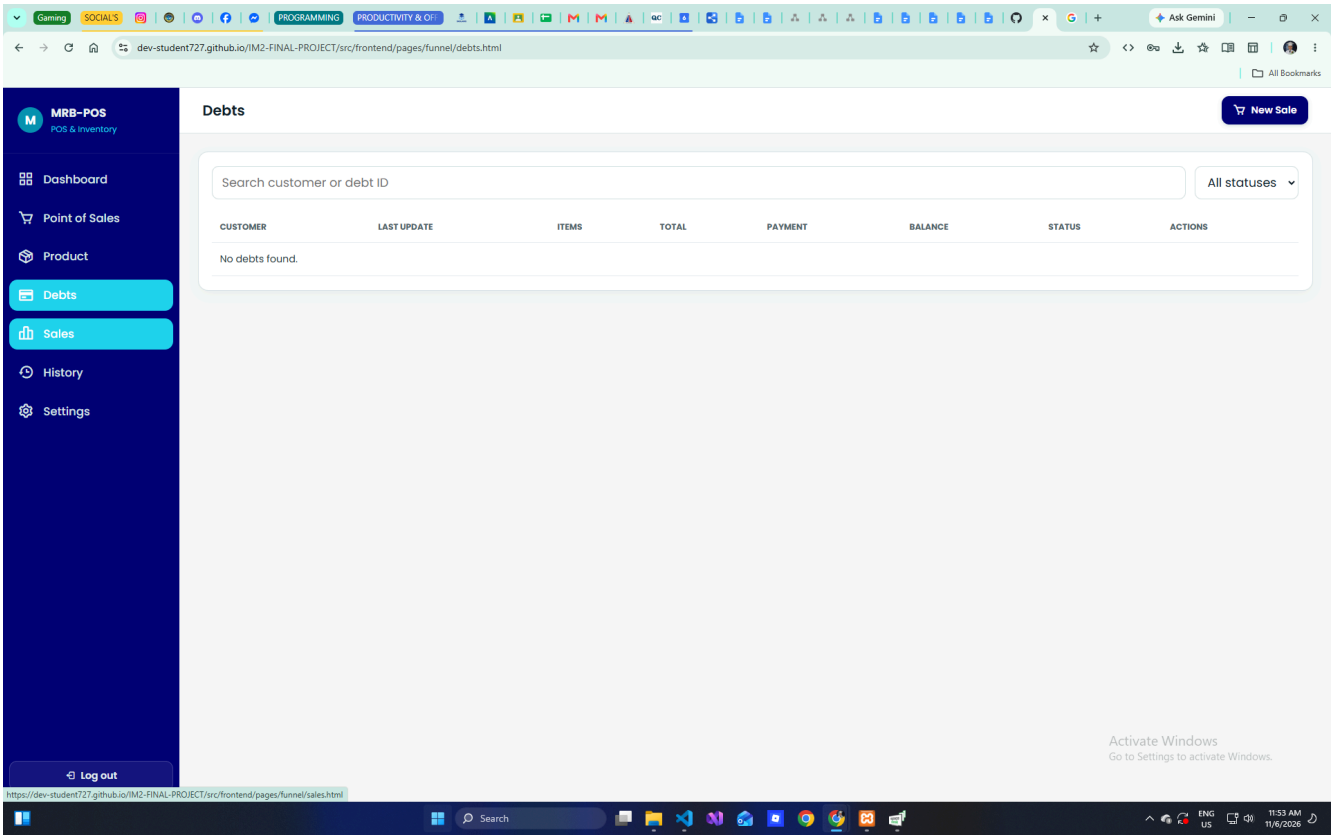


Figure 4.8: History

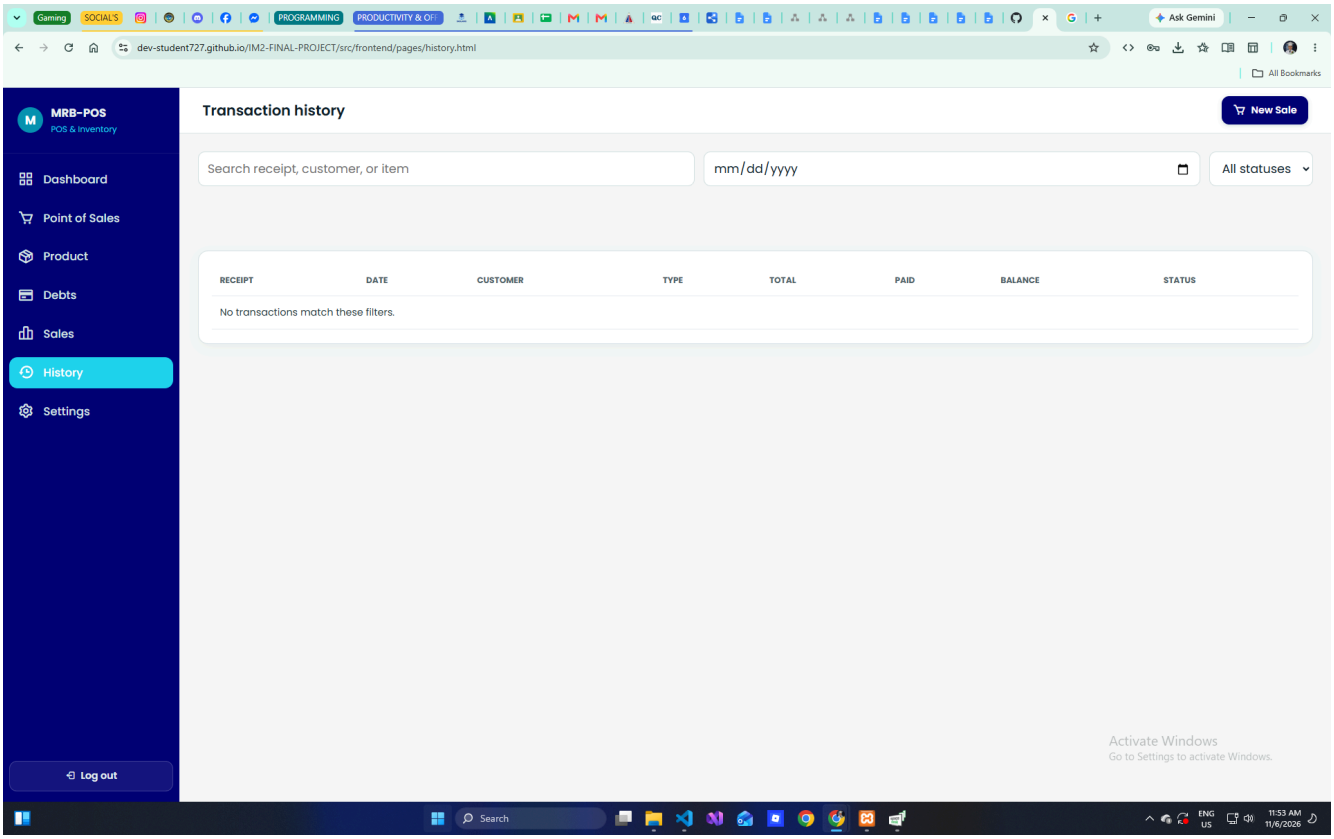
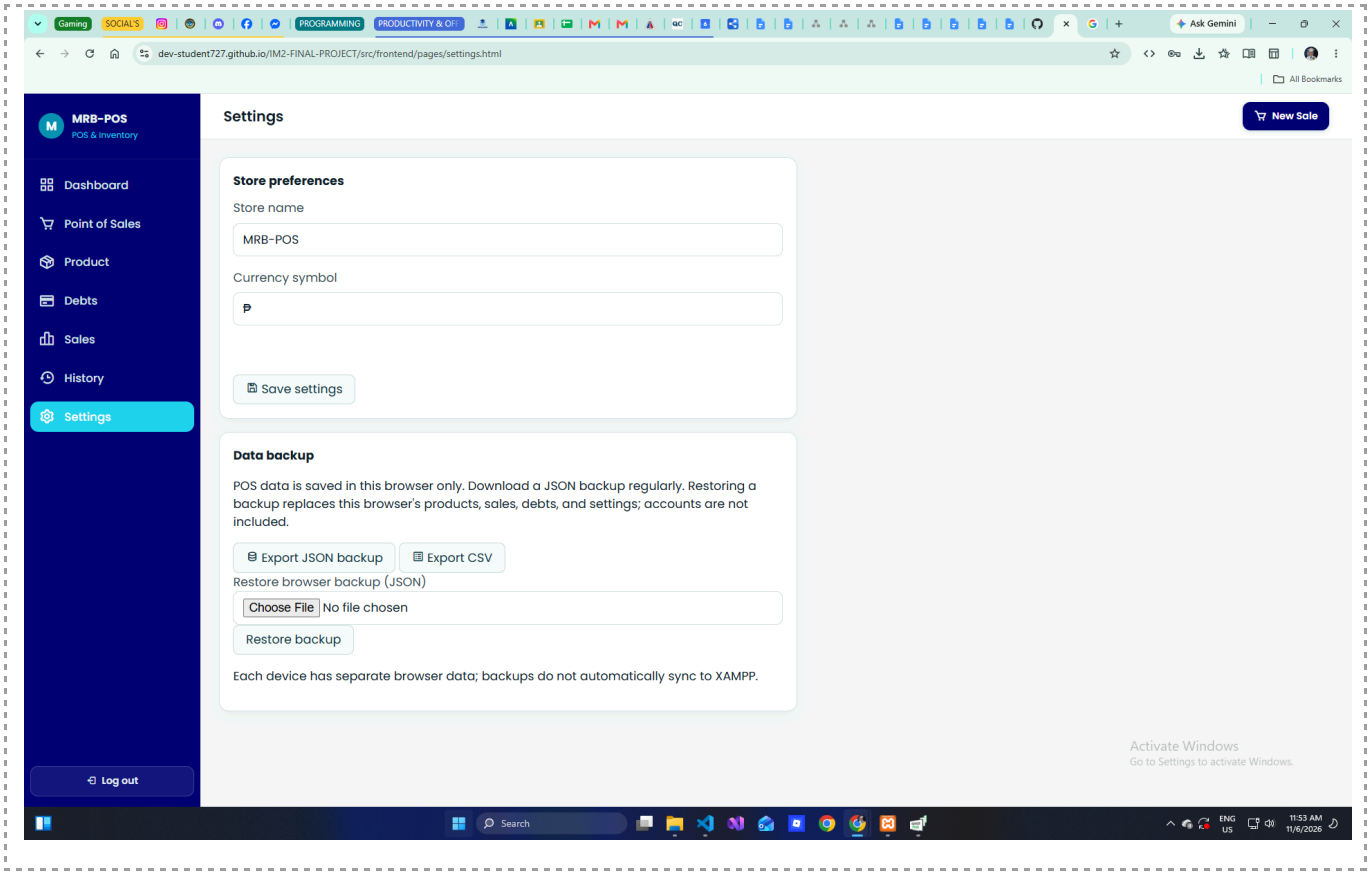


Figure 4.9: Settings



B. Screenshots of the Completed System

Insert at least 3–5 screenshots of your actual running system: login screen, main menu/dashboard, at least one data-entry form, and at least one report/output screen.

Figure 5: Login Screen

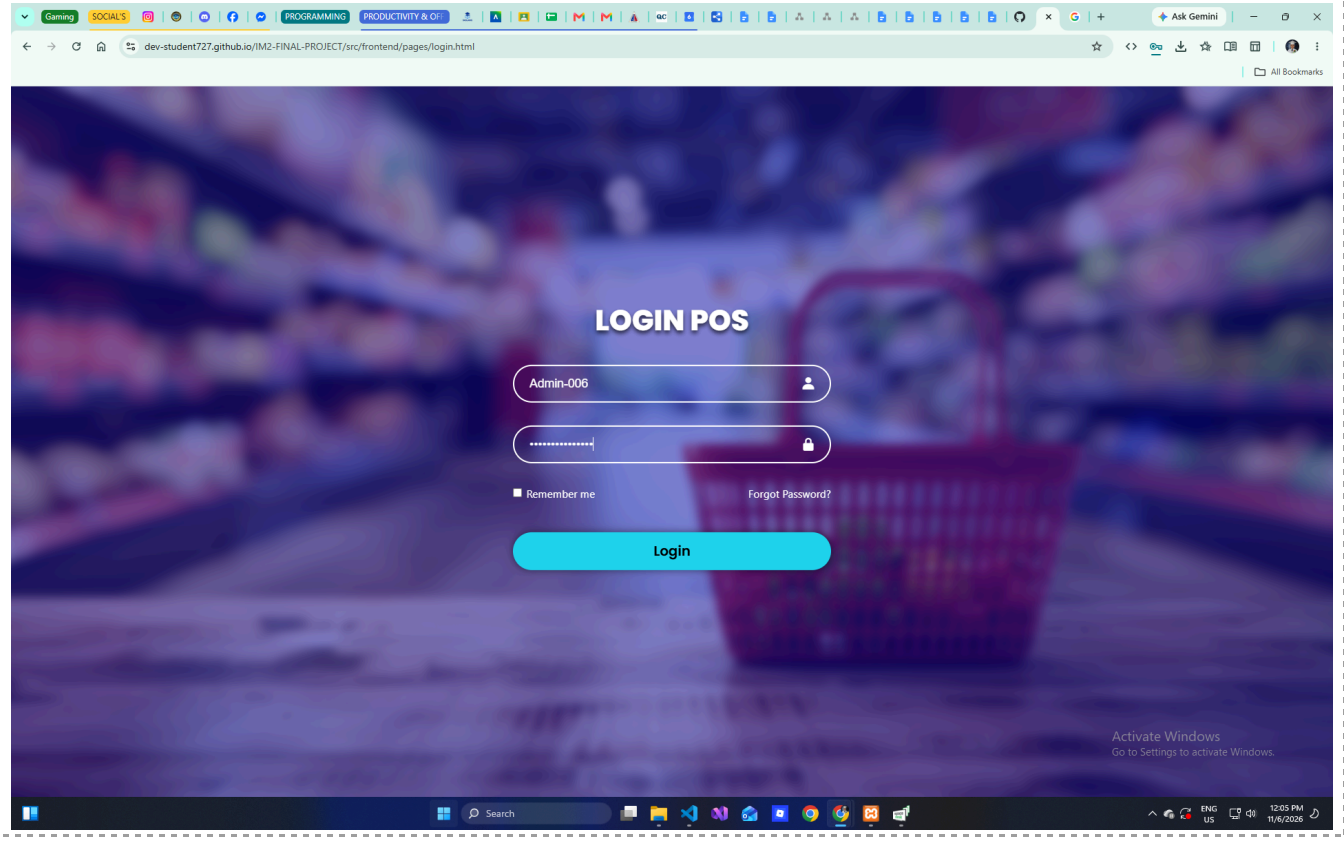


Figure 6: Checkout Product

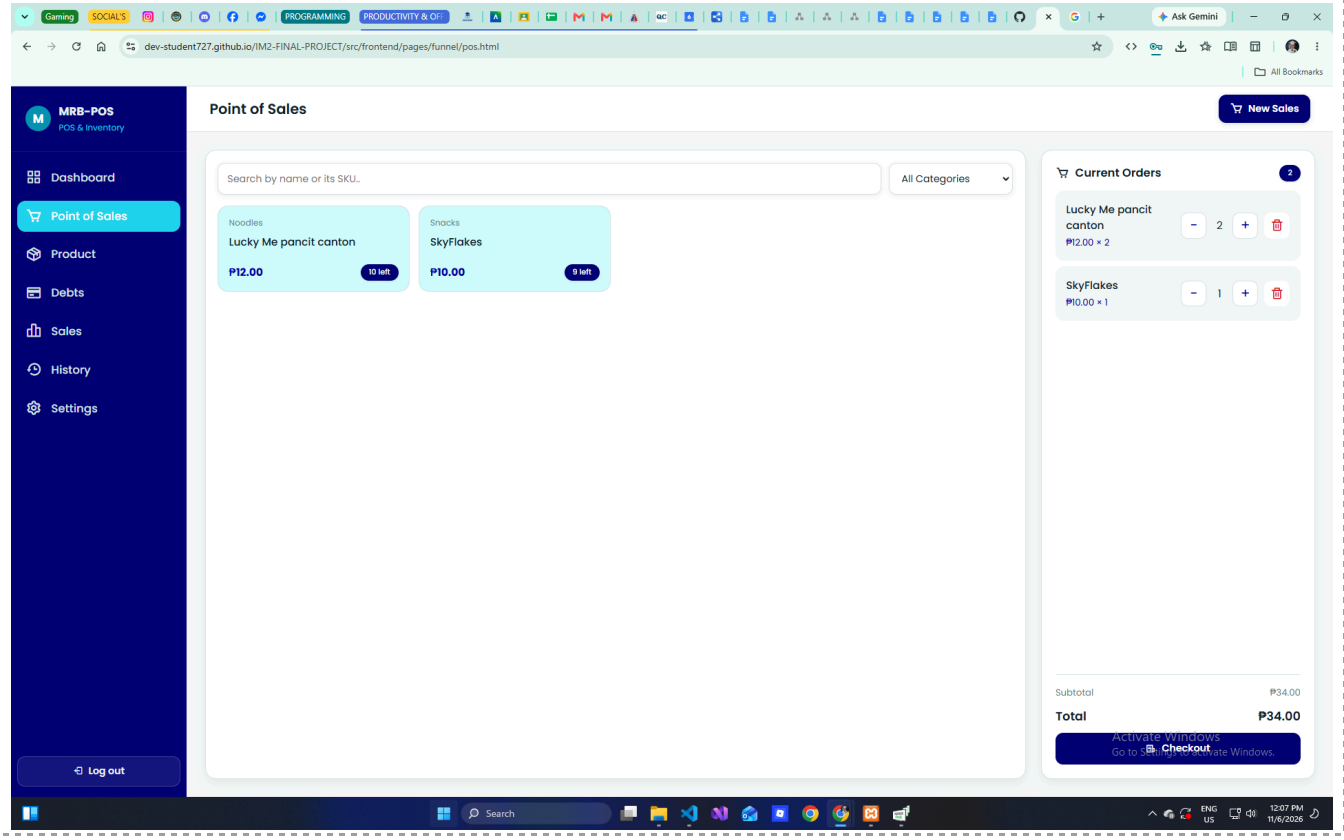
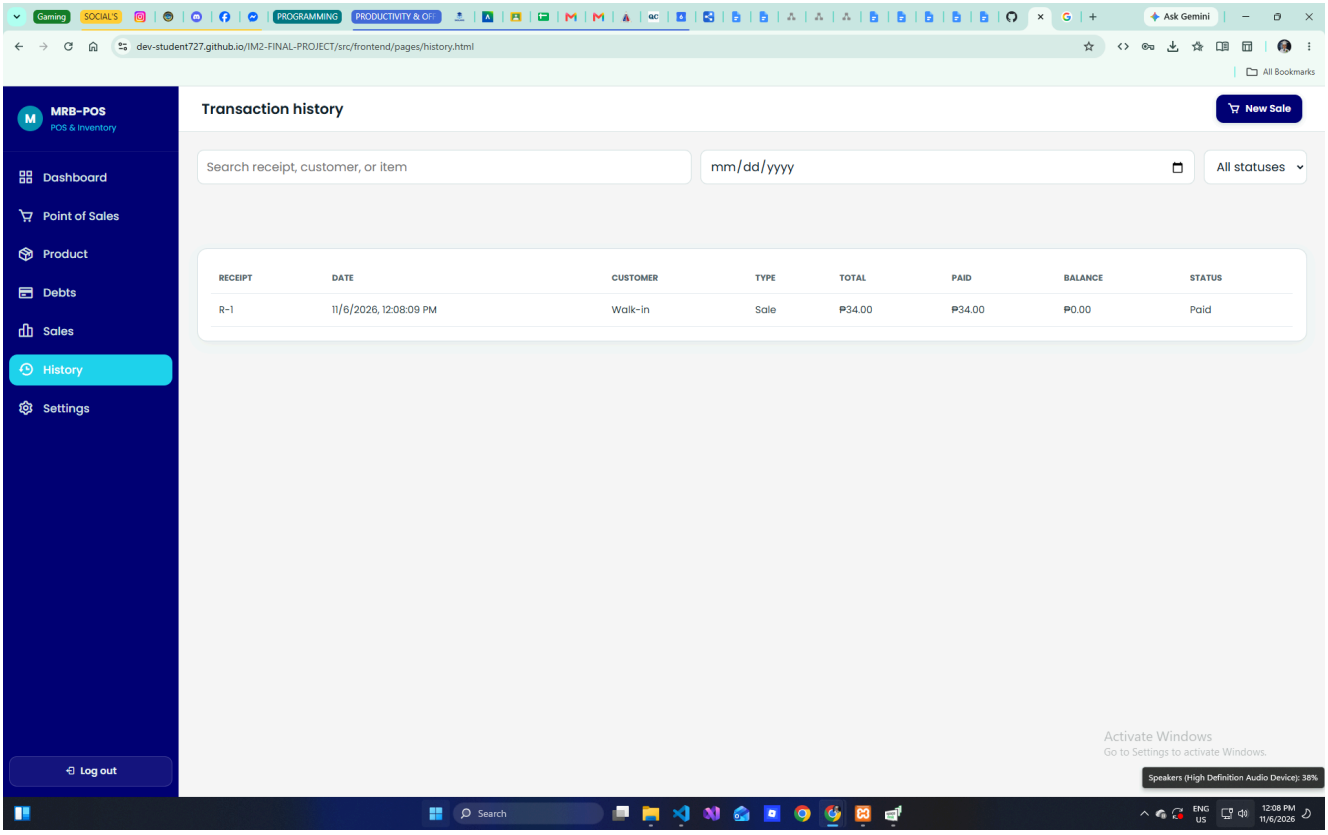


Figure 7: Save Checkout Details



VI. Security and Backup Documentation

A. User Roles and Access Levels

Role	Access / Permissions	Example Use Case
Developer	Full administrative access to system configurations, database tables, source code management, and system-wide reset controls.Managing database schemas, deploying updates, and troubleshooting connection bugs in XAMPP.	Full administrative access to system configurations, database tables, source code management, and system-wide reset controls.Managing database schemas, deploying updates, and troubleshooting connection bugs in XAMPP.
user/owner	Full store-level access to the POS dashboard, product inventory management, sales checkout interface, debt tracking, and data export settings.A sari-sari store owner processing daily customer checkouts, monitoring stock levels, and recording partial debt payments.	Full store-level access to the POS dashboard, product inventory management, sales checkout interface, debt tracking, and data export settings.A sari-sari store owner processing daily customer checkouts, monitoring stock levels, and recording partial debt payments.
customer	Limited read-only access (via display screens or printed/digital receipts) to view their own purchase totals, active credit balances, and payment histories.A regular community customer checking their current outstanding store credit balance or requesting a receipt summary.	Limited read-only access (via display screens or printed/digital receipts) to view their own purchase totals, active credit balances, and payment histories.A regular community customer checking their current outstanding store credit balance or requesting a receipt summary.

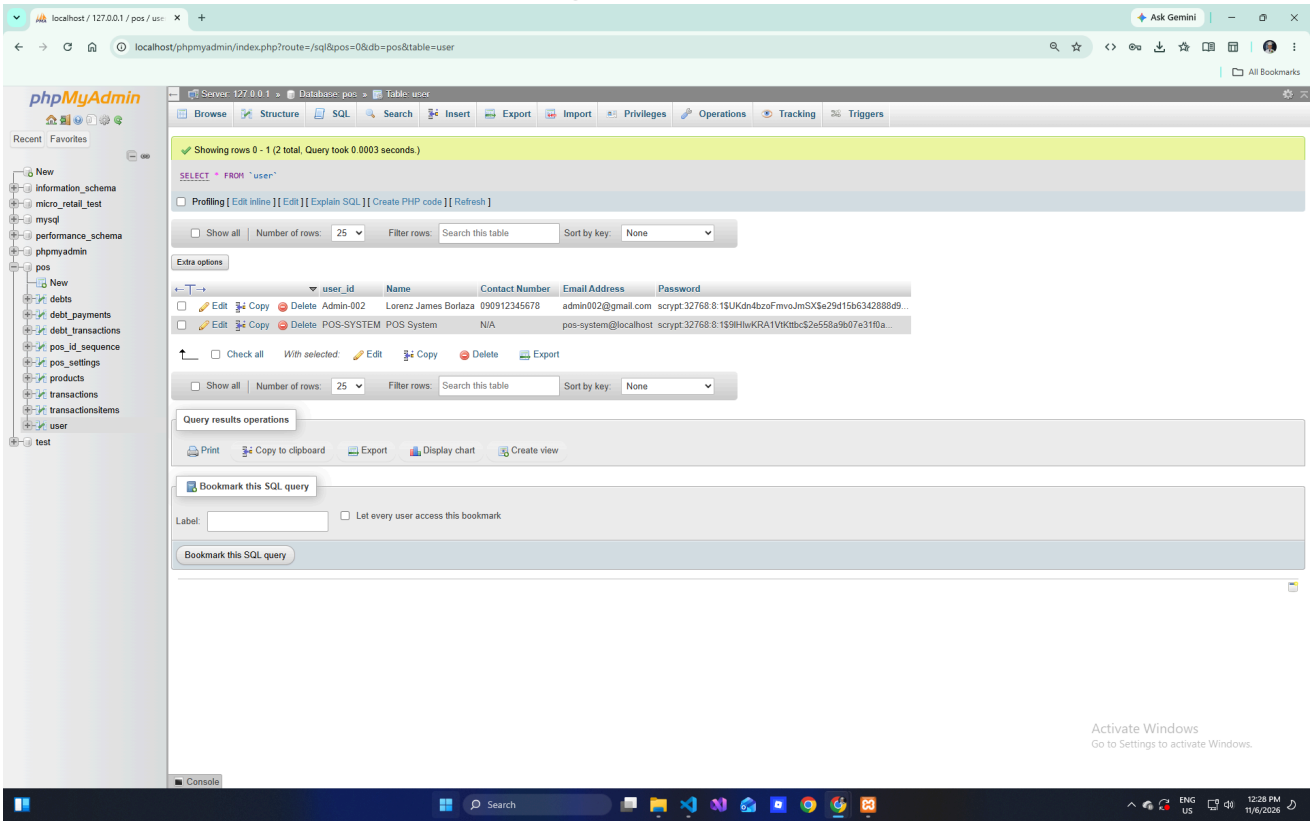
B. Password Hashing Method Used

The password Hashind method used is the built in password hashing of the browser, itself since again my system only runs the backend on local storage, because my xampp didn't read by the backend deployed, so the the password hashing method used is the built in hashing for local browser or storage, but if my systems backend runs on xampp my sql, it will be using a bcrypt password hashing.

C. Backup and Restore Procedure (via phpMyAdmin / XAMPP)

For Backup and restore procedure, i didn't perform a backup and restoration, why? Because i tried to deploy my backend, but in some case, when i already deployed the backend and tried to run my frontend, it does not reach my xampp sql, so that i didn't perform backup, and restoration, but i can still show my xampp that it can fetch data from frontend if configure well, even it is not deployed, just on its localhost only.

Xampp Database Fetching Data from frontend on the local port
[Figure 8: User Fetch]



[Figure 9: TransactionItems]

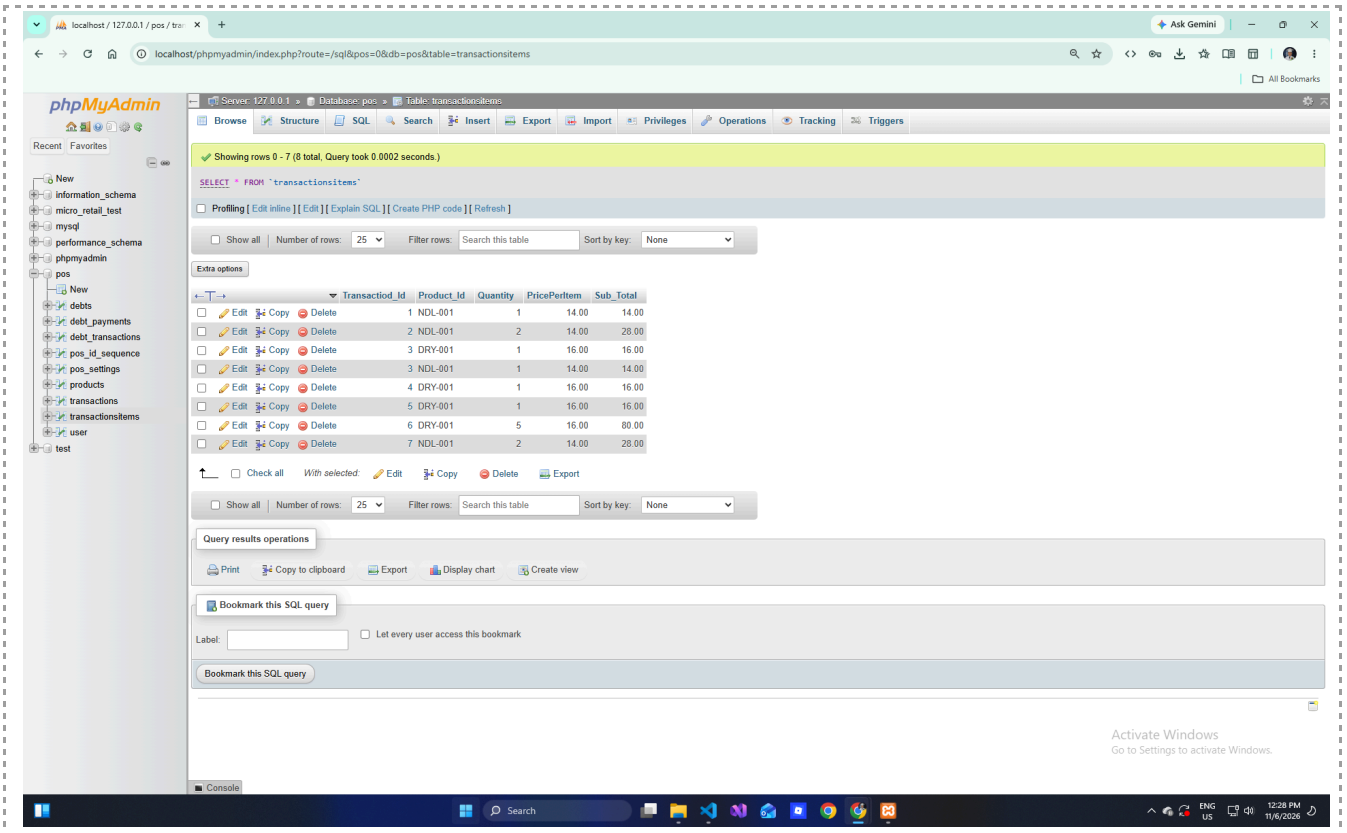


Figure 10: Transactions

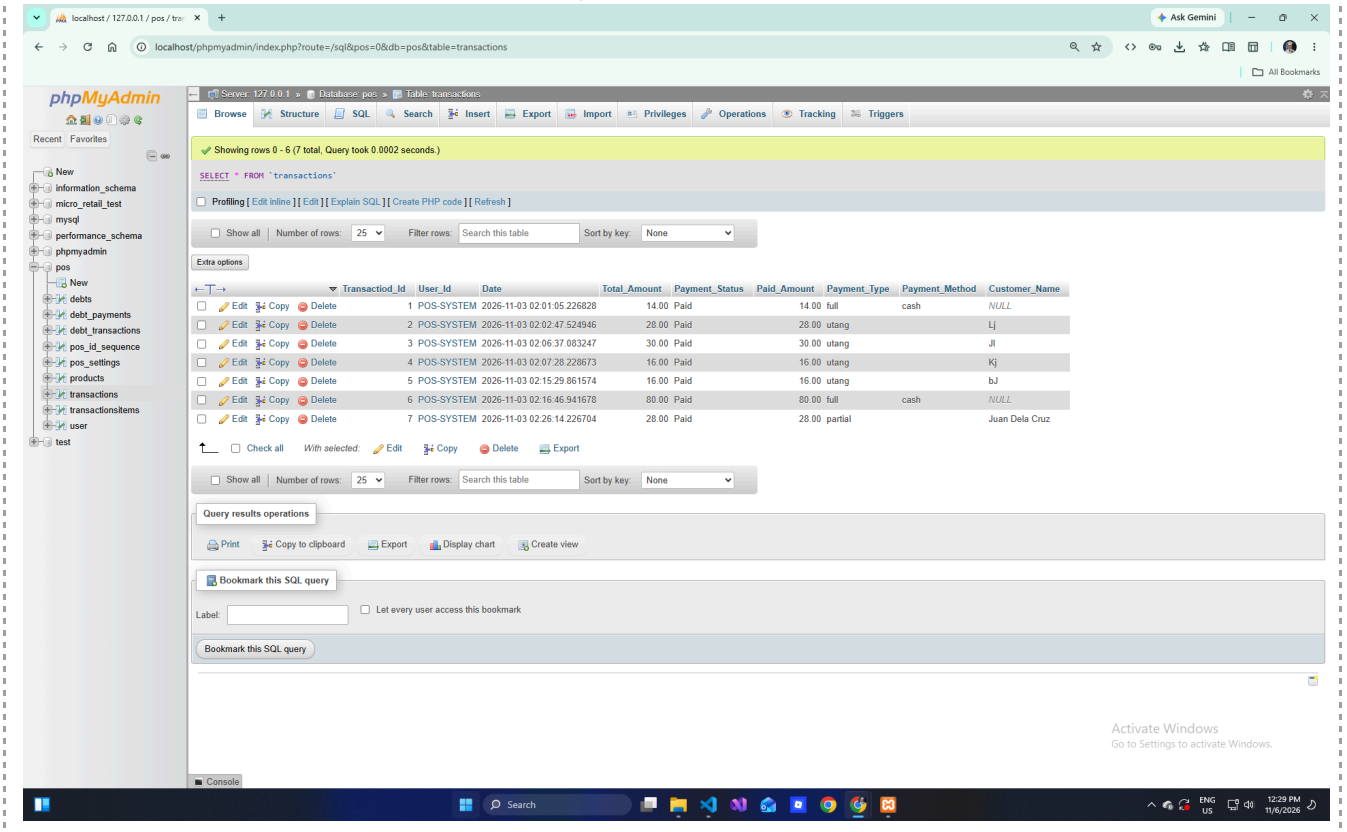


Figure 11: Products

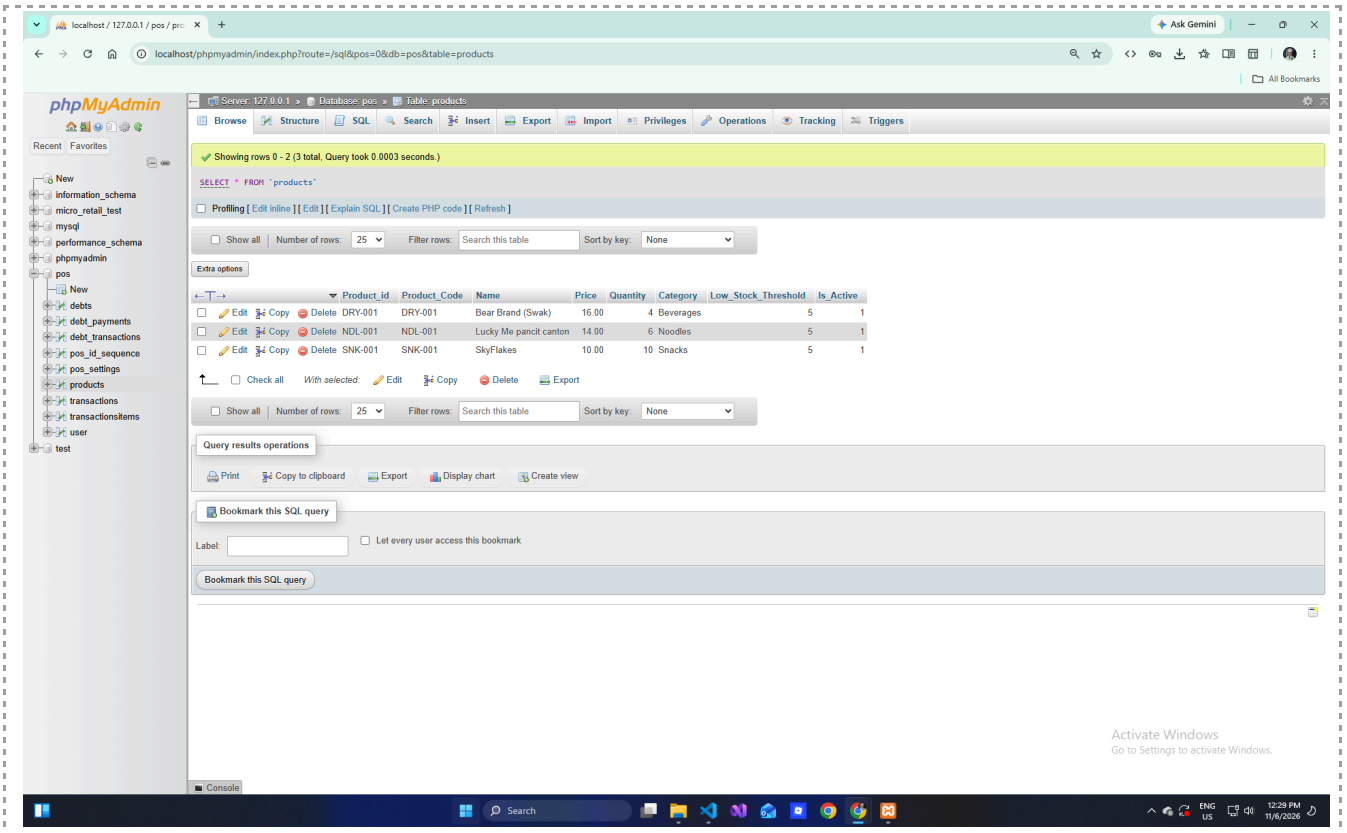


Figure 12: pos_settings

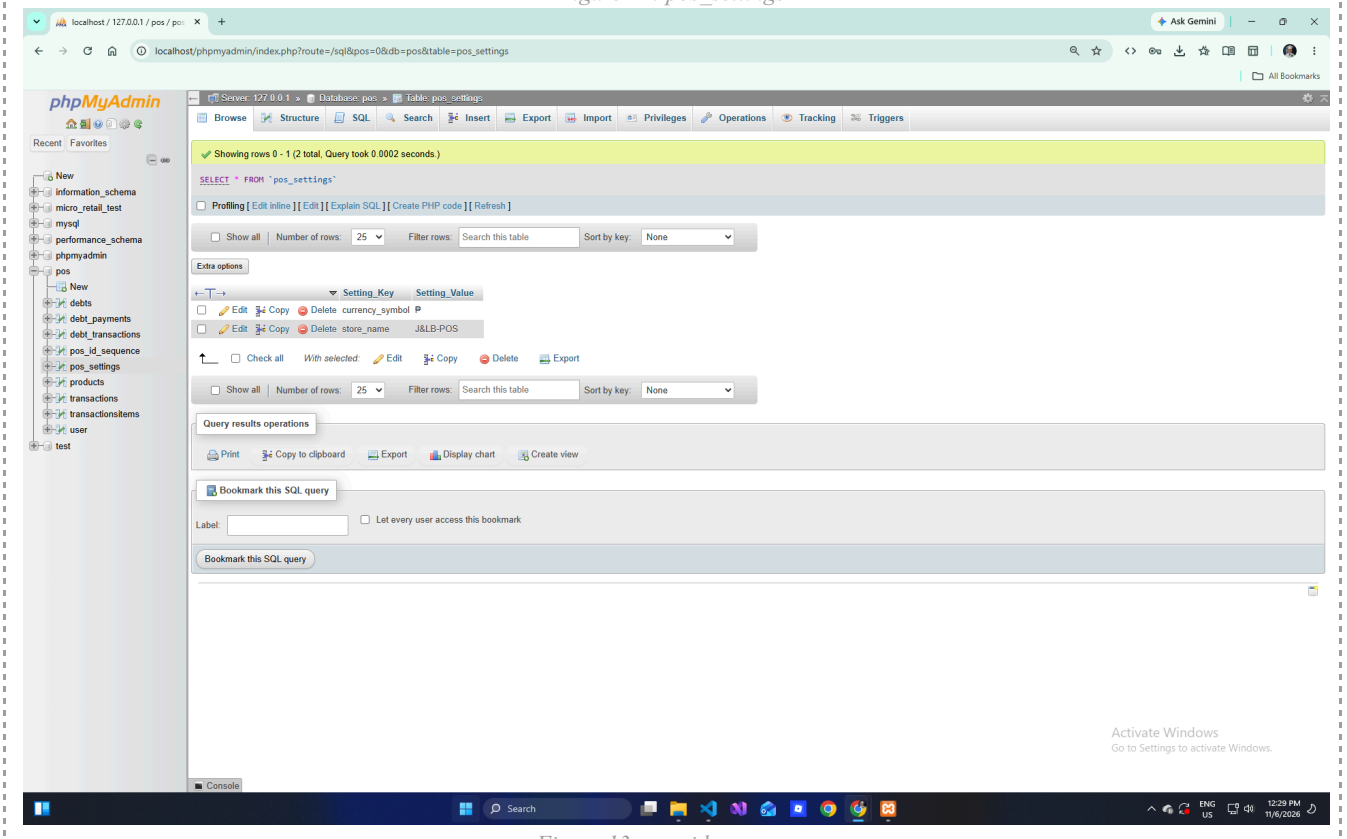


Figure 13: pos_id_sequence

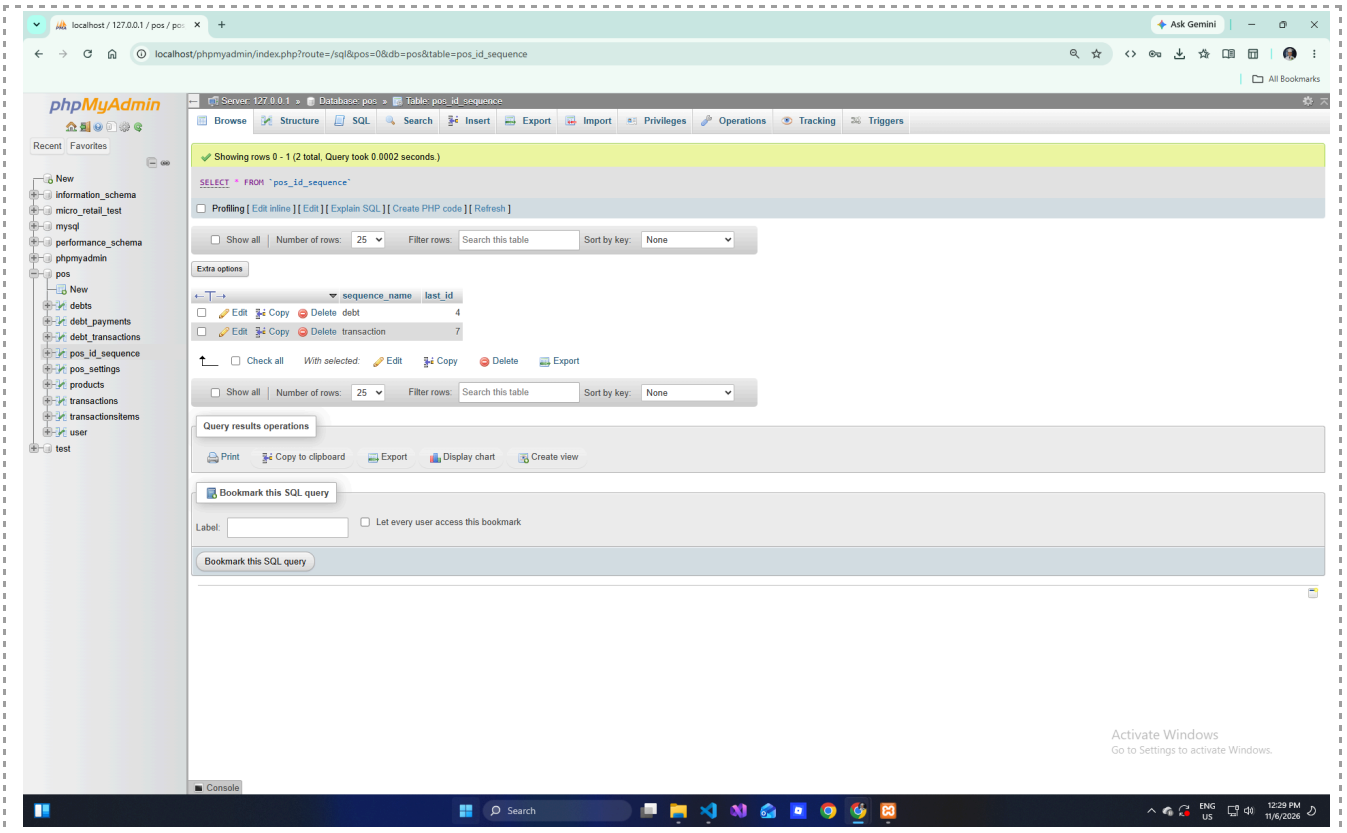


Figure 14: debt_transactions

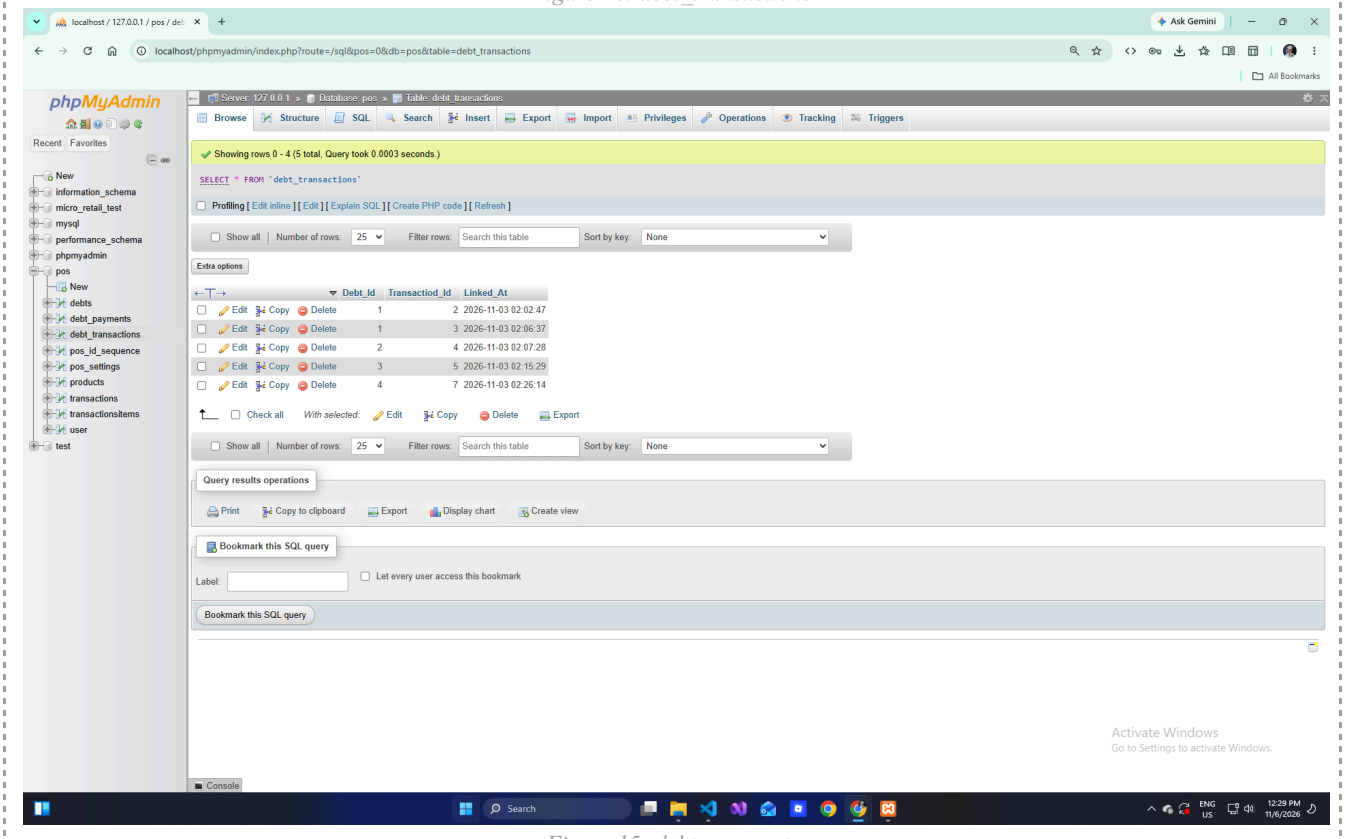


Figure 15: debt_payments

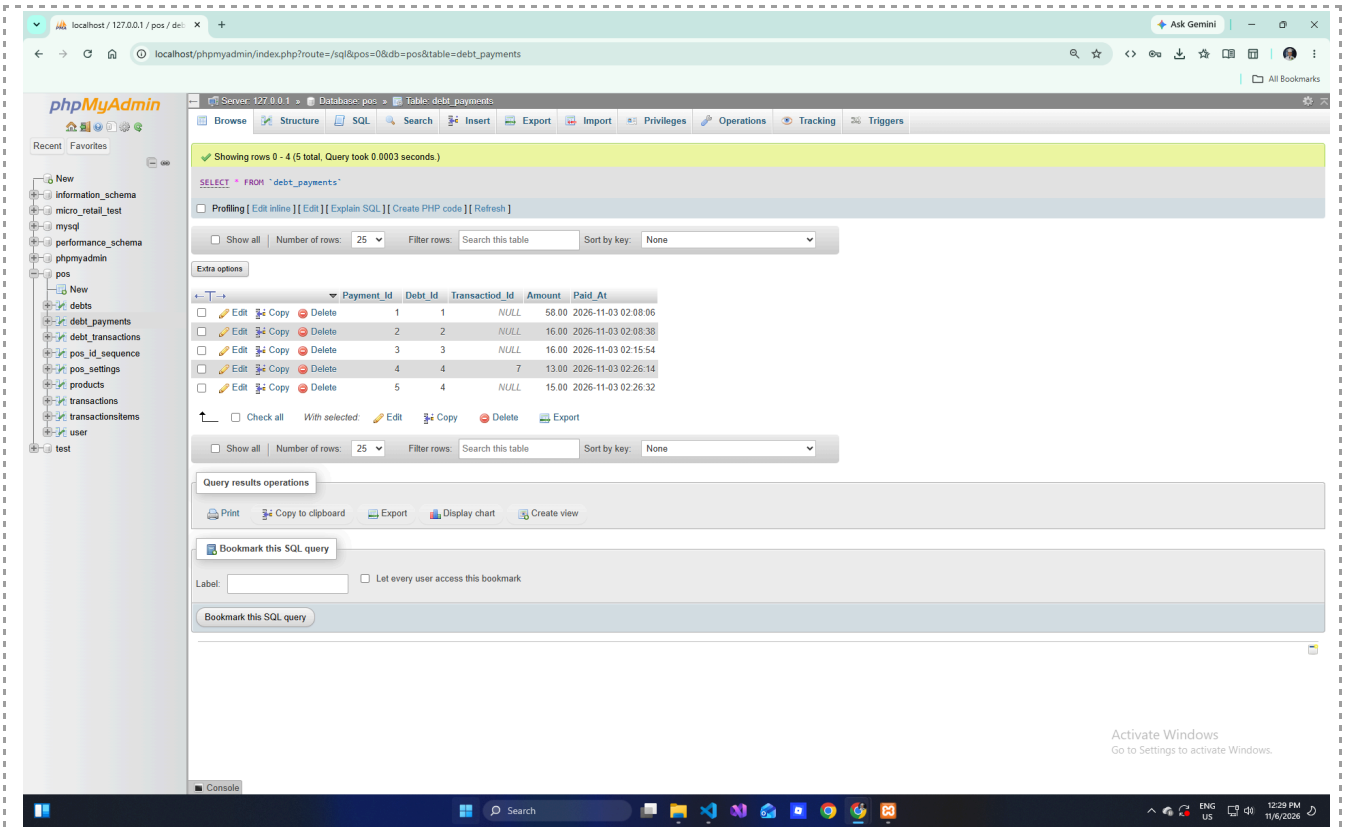
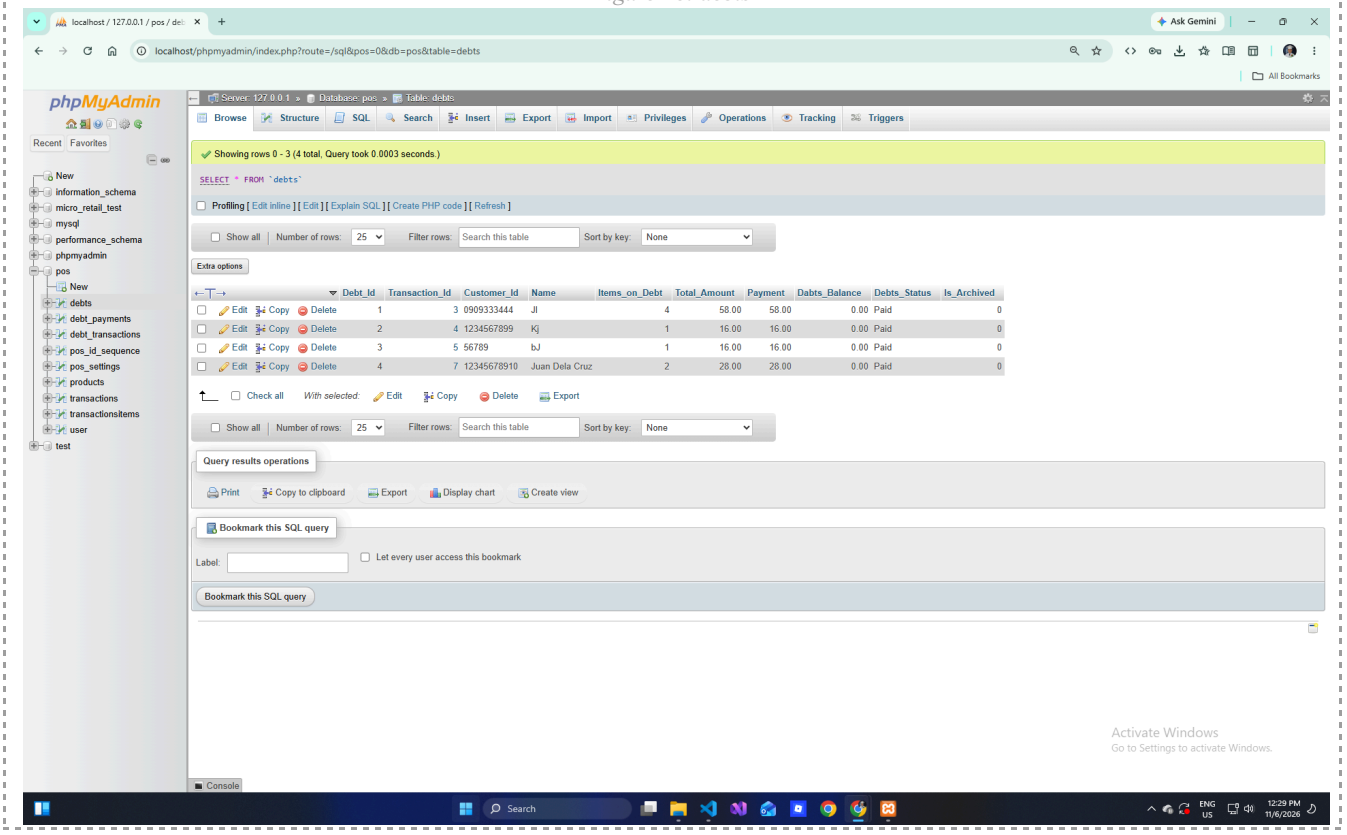


Figure 16: debts



VII. Individual Reflection (Data Privacy and Ethics)

One to two pages. Address: what personal or sensitive data your system stores, how you protected it, and what data-privacy or ethical risks would exist if the system were deployed for real (Data Privacy Act considerations).

So if my system were deployed for real, and inline with the Data privacy act, i would say the my system would store the most crucial and sensitive data in a most protected storage, and i protect it, and know atlead exact data privacy or ethical risk the would exists is that:

First, operating a digital Point-of-Sale (POS) and inventory management system for a neighborhood micro-retail or sari-sari store involves handling specific categories of information that fall under the scope of data privacy guidelines.

Specifically, the system processes and stores user authentication credentials (such as hashed passwords and email identifiers), operational sales logs, and customer credit ledger details (including customer names, contact numbers, outstanding debt balances, and payment histories).

In traditional settings, these records were kept in vulnerable paper notebooks; however, digitizing them transitions this information into a centralized relational database hosted via XAMPP (MySQL/MariaDB).

To ensure this sensitive information is adequately protected, several technical safeguards are integrated into the system's architecture. Passwords are never stored in plain text; instead, they undergo secure cryptographic hashing using industry-standard algorithms before being saved to the user table.

Furthermore, database access is strictly restricted through role-based permissions, separating administrative functions from standard checkout operations. Foreign key constraints and relational normalization (3NF) ensure data integrity, preventing orphaned records or unauthorized modifications across connected tables such as transactions, transactionitems, and debts.

However, deploying this system in a real-world commercial environment introduces notable data privacy and ethical risks aligned with the principles of the Philippine Data Privacy Act of 2012 (Republic Act No. 10173).

The primary ethical concern revolves around the collection and processing of personal identifiable information (PII) from neighborhood customers—particularly regarding customer debt records and contact details. In a close-knit community,

exposing or improperly securing a customer's credit ledger could lead to social stigma, discrimination, or loss of privacy. If unauthorized individuals gained access to the database, customers' financial standing and purchasing habits would be compromised.

And in conclusion of my reflection, all of this saying was probably adopted to my system in the future, not only just a scure storign information, but make if safest storage for my system and lots of security even more, in able to adopt in an ethical and data privacy act of 2012.

VIII. References

List any references (textbooks, articles, official documentation) cited in this paper, in a consistent citation style (e.g., APA).

1. **National Privacy Commission.** (2012). *Republic Act No. 10173: Data Privacy Act of 2012*. Official Gazette of the Republic of the Philippines. https://www.privacy.gov.ph/data-privacy-act/
2. **Silberschatz, A., Korth, H. F., & Sudarshan, S.** (2020). *Database System Concepts* (7th ed.). McGraw-Hill Education.
3. **Elmasri, R., & Navathe, S. B.** (2015). *Fundamentals of Database Systems* (7th ed.). Pearson.
4. **Oracle Corporation.** (2024). *MySQL 8.0 Reference Manual: Security and Password Hashing*. Oracle Documentation. https://dev.mysql.com/doc/refman/8.0/en/
5. **Apache Friends.** (2024). *XAMPP Apache + MariaDB + PHP + Perl Documentation*. Apache Friends Open Source Distribution. https://www.apachefriends.org/docs/

Appendix A — Source Code Repository / Submission Link

GitHub / Google Drive / Other link: https://github.com/dev-Student727/IM2-FINAL-PROJECT

Default test login credentials (for grading): _____

Appendix B — Approval Sheet

This Final Project, entitled "_____," prepared and submitted by _____, in partial fulfillment of the requirements for Information Management 2 (Advance Database System), is hereby approved.

Student Signature over Printed Name	Date
Instructor Signature over Printed Name	Date